

UNIVERSITÀ DEGLI STUDI DI SALERNO

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE ED ELETTRICA E
MATEMATICA APPLICATA



Algoritmi e Protocolli per la Sicurezza

PROJECT WORK

Group Members:

Nome	Matricola	Email
Contaldo Luigi	IE22700111	l.contaldo1@studenti.unisa.it
D'Errico Fabrizio	IE22700121	f.derrico2@studenti.unisa.it

ANNO ACCADEMICO 2025/2026

Indice

1	Introduzione WP1	4	12	Apertura delle Urne	20
2	Attori Coinvolti	4	12.1	Richiesta di Attestazione	20
3	Avversari	6	13	Autenticazione e Rilascio del Token di Ses- sione	20
3.1	Avversario Passivo (Eavesdropper) . .	6	13.1	Generazione della Richiesta Autenticata	20
3.2	Gestore Disonesto dell'Autorità di Raccolta	6	13.2	Invio della Richiesta	21
3.3	Gestore Disonesto dell'Autorità di Scrutinio	6	13.3	Validazione e Controlli Anti-Frode . .	21
3.4	Autorità di Autenticazione Disonesta .	7	13.4	Creazione del Token di Sessione	22
3.5	Avversario Esterno	7	14	Formato della Scheda di Voto e Oper- azioni Crittografiche	22
3.6	Elettore Disonesto e Coercitore	7	14.1	Struttura della Scheda di Voto Cifrata .	22
4	Proprietà di Sicurezza	7	14.2	Operazioni Crittografiche – Dettaglio .	23
4.1	Confidenzialità	7	14.2.1	Cifratura del Voto (Elettore) .	23
4.2	Integrità	8	14.2.2	Verifica della Scheda (AR) . .	25
4.3	Trasparenza	9	15	Chiusura delle Urne	26
4.4	Efficienza	9	15.1	Attestazione di Chiusura	26
5	Mapping Avversari ↔ Proprietà	9	16	Ricostruzione della Chiave e Decifratura	27
6	Completezza	11	16.1	Protocollo di Ricostruzione	28
7	Relazioni di Fiducia	11	16.2	Calcolo e Pubblicazione del Risultato .	29
7.1	Autorità di Autenticazione	11	17	Meccanismi di Verifica	30
7.2	Decifratura del Voto	12	17.1	Verifica Individuale (Elettore)	30
7.3	Urna Elettronica	12	17.2	Verifica Universale	31
7.4	Ente Certificatore Legale	12	17.3	Revocabilità e Sostituzione del Voto . .	32
8	Introduzione WP2	14	17.3.1	Motivazione della Scelta . . .	32
9	Scelte Crittografiche Principali	14	18	Smart Contract VotingContract	33
10	Generazione e Distribuzione delle Chiavi	16	18.1	Ruolo e Perimetro	33
10.1	Flusso Comune di Distribuzione delle Chiavi	16	18.2	Macchina a Stati	33
10.2	Chiavi dell'Autorità di Autenticazione (AA)	16	19	Introduzione WP3	35
10.3	Chiavi dell'Autorità di Raccolta (AR) .	17	20	Analisi per Avversario	35
10.4	Chiavi dell'Autorità di Scrutinio (AS) – Schema Threshold	17	20.1	Avversario Passivo (Eavesdropper) . .	35
10.5	Chiavi dei Garanti	17	20.2	Gestore Disonesto dell'Autorità di Raccolta (AR)	35
10.6	Chiavi dell'Elettore	18	20.3	Gestore Disonesto dell'Autorità di Scrutinio (AS)	36
11	Registrazione degli Elettori	18	20.4	Autorità di Autenticazione Disonesta (AA)	37
11.1	Generazione delle Chiavi Locali	18	20.5	Avversario Esterno	37
11.2	Richiesta di Identificazione	19	20.6	Elettore Disonesto e Coercitore	38
11.3	Validazione e Emissione del Certificato	19			
11.4	Conclusione della Procedura	19			

21	Analisi per Proprietà	39		
21.1	Confidenzialità	39		
21.2	Integrità	40		
21.3	Trasparenza	41		
21.4	Efficienza	42		
22	Analisi Globale del Sistema	42		
22.1	Assunzioni di Fiducia Residue	42		
22.2	Conflitti tra Proprietà	43		
22.3	Sintesi	44		
23	Strumenti Crittografici Utilizzati	45		
23.1	RSA-2048 e PKI X.509	45		
23.2	RSA-OAEP (Cifratura del Voto)	45		
23.3	RSA-PKCS1v15 + SHA-256 (Firma Digitale)	45		
23.4	SHA-256 (Hash e Commitment)	45		
23.5	AES-256-GCM (Cifratura Simmetrica delle Share)	45		
23.6	Schema di Shamir ($t=2, n=3$) su $GF(p)$	46		
24	Architettura Software: Ruoli degli Script Python	46		
25	Analisi dei Tempi di Esecuzione	47		

DISCLAIMER

Il presente documento costituisce il Project Work per il corso di Algoritmi e Protocolli per la Sicurezza (A.A. 2025–2026), tenuto dai docenti Carlo Mazzocca e Francesco Cauteruccio. Il lavoro qui presentato è frutto dell'ingegneria didattica: le scelte progettuali, i compromessi e le analisi sono state effettuate con l'obiettivo di dimostrare la comprensione critica degli strumenti crittografici e dei modelli di sicurezza studiati durante il corso.

Nessuna delle soluzioni proposte è da intendersi pronta per un impiego in scenari reali, poiché la sicurezza in produzione richiederebbe ulteriori audit formali, analisi di vulnerabilità implementative e compliance normativa (es. GDPR, Regolamento elettorale vigente), aspetti che esulano dagli obiettivi formativi del presente progetto. Le assunzioni di fiducia dichiarate nei confronti delle entità del sistema sono esplicitate e motivate, e pertanto eventuali violazioni di tali assunzioni possono compromettere le garanzie descritte. Il lettore è invitato a considerare il lavoro come un esercizio critico di progettazione e non come un sistema produttivo.

WORK PACKAGE 1 — Modello di Sicurezza e Requisiti

1 Introduzione WP1

Il presente work package definisce il modello di sicurezza del sistema, identificando gli attori coinvolti, le loro responsabilità e le relazioni di fiducia reciproca. Il sistema è progettato per un referendum binario (Sì/No); questa scelta permette di bilanciare efficacia operativa e rigore crittografico, pur garantendo i requisiti necessari a qualsiasi sistema di voto elettronico sicuro.

Il sistema si fonda sul **principio di separazione dei poteri**: nessuna singola entità deve detenere il controllo completo sull'intero ciclo di vita del voto (autenticazione, espressione, raccolta, scrutinio).

Motivazioni della scelta binaria

- **Semplicità crittografica**: il voto $\in \{0, 1\}$ binario consente di rappresentare ogni preferenza come un singolo bit e rende immediato il calcolo del risultato dopo la fase di scrutinio, senza introdurre meccanismi crittografici più complessi.
- **Verificabilità**: il risultato (somma totale) è facilmente controllabile da chiunque.
- **Estensibilità**: l'estensione a elezioni con lista chiusa o preferenza nominale resta possibile come sviluppo futuro.

2 Attori Coinvolti

Il sistema di voto elettronico coinvolge le seguenti entità, ciascuna con responsabilità ben definite e separate.

Table 1: Attori coinvolti nel sistema di voto elettronico

Attore	Ruolo / Responsabilità
Elettore	Soggetto avente diritto al voto che interagisce con il sistema in quattro fasi distinte: registrazione, autenticazione, espressione del voto e verifica a posteriori della corretta inclusione del voto nel conteggio.
Autorità di Autenticazione (AA)	Entità preposta alla distribuzione delle credenziali di accesso al sistema e alla verifica dell'identità degli elettori, garantendo l'unicità del voto. Il suo ruolo termina nel momento in cui l'elettore effettua l'accesso al portale.
Autorità di Raccolta (AR)	Entità che riceve e conserva i voti espressi durante la finestra di votazione, operando esclusivamente su dati cifrati. Rilascia all'elettore una ricevuta per la verifica individuale. I meccanismi anti-duplicazione (token monouso, anti-doppio voto) sono applicati dallo smart contract on-chain, non dall'AR in modo autonomo.
Autorità di Scrutinio (AS)	Entità che, al termine della votazione, recupera i voti cifrati direttamente dalla blockchain tramite il registro pubblico VotingContract, li decifra, ne calcola il risultato e lo pubblica insieme alle prove necessarie alla verificabilità universale.
Ente Certificatore Legale (EC)	Terza parte istituzionalmente indipendente, nominata da un organo con valore legale. Firma le attestazioni di apertura delle urne ($Att_{apertura}$), di chiusura ($Att_{chiusura}$) e di risultato ($Att_{risultato}$). Registra direttamente on-chain apertura e chiusura invocando le funzioni <code>openElection</code> e <code>closeElection</code> ; l'attestazione di risultato, invece, viene allegata da AS alla pubblicazione finale tramite <code>publishResult</code> . Il suo ruolo è esclusivamente legale-certificativo: non partecipa alla crittografia dei voti né al conteggio.

Ethereum come infrastruttura di registro

Il protocollo integra la rete **Ethereum** come livello di registro pubblico immutabile. Ethereum non è un attore nel senso tradizionale (non ha obiettivi propri né può agire in modo autonomo), ma costituisce l'infrastruttura su cui le autorità (AR, AS, EC) pubblicano i dati critici del processo elettorale tramite uno **smart contract dedicato** (VotingContract). Il contratto è deployato e verificabile pubblicamente; il suo indirizzo è pubblicato nella documentazione ufficiale dell'elezione prima dell'apertura delle urne. Le garanzie offerte da Ethereum sono: immutabilità dei dati scritti, assenza di un punto di controllo centralizzato, e verificabilità pubblica senza credenziali.

3 Avversari

Accanto agli attori onesti, il modello di minaccia considera le seguenti figure avversarie, ciascuna caratterizzata da risorse e obiettivi specifici.

3.1 Avversario Passivo (Eavesdropper)

- **Obiettivi:** Intercettare il traffico di rete tra elettore e Autorità di Raccolta per correlare metadati (indirizzi IP, timestamp, dimensione e frequenza dei messaggi) con l'identità dell'elettore, violando il pseudo-anonimato senza alterare alcun messaggio. Non mira a modificare i dati, ma a ricavare informazioni sul voto espresso.
- **Risorse:** Accesso in lettura al canale di comunicazione; capacità computazionali generiche per l'analisi statistica del traffico. Non dispone di privilegi amministrativi sui server né di chiavi crittografiche.

3.2 Gestore Disonesto dell'Autorità di Raccolta

- **Obiettivi:** Alterare, sopprimere o introdurre voti fraudolenti nel database. Tentare di violare il pseudo-anonimato analizzando metadati (timestamp, indirizzi IP, pattern di accesso).
- **Risorse:** Ha accesso amministrativo al server di raccolta, ma non dispone della chiave di decifratura dei voti.
- **Vettori di attacco specifici all'integrazione blockchain:** Con l'introduzione dello smart contract, AR dispone di nuovi vettori di attacco da considerare nel modello di minaccia:
 1. **Omissione selettiva di `submitBallotAnchor`:** AR potrebbe accettare e registrare nel database off-chain i voti di alcuni elettori, ma omettere deliberatamente di invocare `submitBallotAnchor` per quelle schede. I voti sarebbero presenti nell'urna off-chain ma invisibili on-chain: gli elettori interessati non potrebbero verificare l'inclusione del proprio voto tramite la chain, e i verificatori universali troverebbero una discrepanza tra il numero di `ballotAnchors` on-chain e la dimensione dell'urna trasmessa ad AS. Questa asimmetria è rilevabile confrontando il conteggio degli ancoraggi on-chain con N dichiarato in chiusura, ma richiede che i verificatori eseguano tale confronto attivamente.
 2. **Ancoraggio di schede fittizie:** AR potrebbe invocare `submitBallotAnchor` per schede mai ricevute da elettori reali (attacco mitigato dalla guardia `require(usedTokens[_tokenHash])` nel contratto, che impone la previa esistenza di un consumo di token regolare).

3.3 Gestore Disonesto dell'Autorità di Scrutinio

- **Obiettivi:** Tentare di decifrare voti individuali per violare la segretezza o alterare il risultato finale.
- **Risorse:** Dispone delle chiavi di decifratura (o di una share di esse). La sua capacità di azione è limitata se è in atto un meccanismo di *threshold decryption*.

3.4 Autorità di Autenticazione Disonesta

- **Obiettivi:** Rilasciare credenziali a soggetti non aventi diritto (voti fraudolenti), a elettori legittimi per un voto aggiuntivo (doppio voto) e inviare le credenziali di un elettore a un'altra persona, al fine di evitare che l'elettore effettivo voti o per permettere il voto per conto dei non votanti. Tracciare quali elettori hanno ottenuto le credenziali per dedurre informazioni sulle preferenze. Esercitare un *denial-of-service* selettivo rifiutando deliberatamente il rilascio di credenziali a specifici elettori legittimi.
- **Vettore aggiuntivo – emissione multipla di token:** un'AA (o AR) disonesta potrebbe tentare di autenticare più volte lo stesso elettore – con nonce e timestamp freschi a ogni richiesta – per ottenere più token validi da cedere a terzi o da usare per esprimere voti aggiuntivi. Questo vettore è mitigato dal mapping `issuedTokens` on-chain: la funzione `issueToken` del contratto rifiuta qualsiasi seconda emissione per lo stesso $H(Cert_e)$, rendendo il tentativo pubblicamente tracciabile sulla blockchain. Un'AR disonesta che omettesse di invocare `issueToken` prima di generare un token emetterebbe una credenziale non registrata on-chain, che risulterebbe priva di ancoraggio verificabile e rilevabile in sede di audit.

3.5 Avversario Esterno

- **Obiettivi:** Intercettare, modificare o replicare messaggi; impersonare un elettore legittimo rubandone le credenziali; lanciare attacchi *denial-of-service* contro le infrastrutture del sistema.
- **Risorse:** Capacità computazionali generiche e accesso di rete, ma non privilegi amministrativi sui server.

3.6 Elettore Disonesto e Coercitore

- **Obiettivi:** Votare più volte, rivendere il proprio voto dimostrando la preferenza espressa, o falsificare prove del proprio voto (Elettore disonesto). Costringere l'elettore a votare in un dato modo o a fornire una prova del voto espresso (Coercitore).
- **Risorse:** Dispone esclusivamente delle proprie credenziali di autenticazione e dell'accesso all'interfaccia di voto; non detiene privilegi amministrativi sui server né accesso ai dati degli altri elettori. Il coercitore può operare in presenza fisica dell'elettore durante la sessione di voto, sfruttando l'assenza di un ambiente fisicamente controllato tipico del voto elettronico remoto.

4 Proprietà di Sicurezza

Nel presente sottoparagrafo vengono esaminate le principali proprietà che il sistema di voto elettronico deve possedere per garantire un'elezione sicura, trasparente e affidabile. Le proprietà sono analizzate in base ai quattro pilastri fondamentali: confidenzialità, integrità, trasparenza ed efficienza.

4.1 Confidenzialità

Le proprietà di confidenzialità garantiscono che il voto espresso rimanga segreto e che l'identità dell'elettore non possa essere collegata alla sua preferenza.

- C.1:** La preferenza espressa da ciascun elettore deve rimanere segreta e non deve essere rivelabile a nessun'altra parte, nemmeno combinando le informazioni in possesso dei gestori del sistema.
- C.2:** Nessuna singola entità deve essere in grado di associare direttamente l'identità reale dell'elettore alla sua preferenza di voto (*pseudo-anonimato*).
- C.3:** I dati personali e identificativi degli elettori devono essere accessibili esclusivamente all'Autorità di Autenticazione e non devono essere comunicati all'Autorità di Raccolta né all'Autorità di Scrutinio.
- C.4:** Le comunicazioni tra l'elettore e le entità del sistema devono essere protette da intercettazione, in modo che un avversario esterno non possa carpire la preferenza espressa né le credenziali di voto.
- C.5:** La credenziale di voto rilasciata all'elettore non deve contenere informazioni che permettano di risalire all'identità dell'elettore una volta terminata la fase di autenticazione.
- C.6:** Deve essere garantito che un elettore non possa dimostrare a terzi la propria preferenza (resistenza alla coercizione e alla vendita del voto), salvo compromessi progettuali esplicitamente dichiarati.
- C.7:** Il sistema non supporta revoca né sostituzione del voto dopo la registrazione. Tale scelta è un **compromesso progettuale esplicito**: un meccanismo di revoca consentirebbe a un coercitore di imporre all'elettore di annullare e riesprimere il voto sotto controllo, vanificando la resistenza alla coercizione (C.6). Analogamente, la sostituzione del voto renderebbe praticabile la vendita del voto. Il costo ricade sull'elettore: un errore è irrecuperabile dopo la trasmissione. A mitigazione, l'interfaccia richiede una conferma esplicita prima dell'invio definitivo. Si noti che questa proprietà è correlata anche all'integrità: impedire la sostituzione del voto tutela le proprietà I.1 e I.3, evitando che un voto già espresso possa essere rimpiazzato fraudolentemente.

4.2 Integrità

Le proprietà di integrità assicurano che i voti registrati non possano essere alterati, che solo gli aventi diritto possano votare e che il risultato rifletta fedelmente le preferenze espresse.

- I.1:** I voti espressi non devono poter essere alterati, soppressi o duplicati dopo la loro registrazione.
- I.2:** Solo gli elettori aventi diritto, autenticati tramite credenziali verificabili, possono esprimere un voto valido.
- I.3:** Ciascun elettore può esprimere al massimo un voto; tentativi di voto multiplo devono essere rilevati e bloccati.
- I.4:** Il risultato finale deve corrispondere fedelmente e matematicamente ai voti validi espressi.
- I.5:** Le credenziali di voto devono essere autentiche, non falsificabili e non riutilizzabili.
- I.6:** Non deve essere possibile introdurre voti fraudolenti (voti da parte di soggetti non aventi diritto) nel conteggio finale.
- I.7:** Ogni operazione nel sistema (autenticazione, espressione del voto, scrutinio) deve poter essere verificata a posteriori senza ambiguità.

4.3 Trasparenza

Le proprietà di trasparenza garantiscono che il processo elettorale sia pubblicamente verificabile e che le assunzioni di fiducia siano esplicitamente dichiarate.

- T.1:** Il protocollo crittografico utilizzato per il voto deve essere pubblicamente noto, documentato e basato su algoritmi standard e aperti.
- T.2:** Il risultato finale deve essere pubblicamente verificabile da chiunque (*verificabilità universale*), senza richiedere chiavi segrete o privilegi di accesso.
- T.3:** Ogni elettore deve poter verificare autonomamente che il proprio voto sia stato correttamente incluso nel conteggio (*verificabilità individuale*).
- T.4:** Il sistema non deve affidarsi eccessivamente a parti fidate; ogni assunzione di fiducia verso una componente deve essere esplicitamente dichiarata.
- T.5:** Le operazioni di apertura e chiusura delle urne digitali devono essere attestate dall'**Ente Certificatore Legale (EC)**, terza parte indipendente con valore legale riconosciuto.

4.4 Efficienza

Le proprietà di efficienza garantiscono che il sistema sia utilizzabile in pratica, con tempi di risposta ragionevoli e requisiti computazionali contenuti.

- E.1:** L'espressione del voto deve essere completata in tempi ragionevoli (pochi minuti) dal punto di vista dell'elettore.
- E.2:** La verifica individuale deve essere semplice, rapida e accessibile anche a un elettore senza competenze tecniche.
- E.3:** La verifica universale deve poter essere eseguita con risorse computazionali ordinarie.
- E.4:** La dimensione dei messaggi scambiati e delle prove crittografiche allegate deve essere gestibile dalle infrastrutture di rete e di storage disponibili.

5 Mapping Avversari ↔ Proprietà

La Tabella 2 esplicita il collegamento logico tra ciascun avversario identificato e le proprietà di sicurezza che il suo comportamento mira a compromettere. Per ogni coppia viene indicato l'effetto concreto di un attacco riuscito.

Table 2: Mapping tra avversari e proprietà minacciate

Avversario	Proprietà minacciate	Effetto dell'attacco
Gestore disonesto dell'AR	Integrità (I.1, I.4), Confidenzialità (C.2, C.5)	Alterazione o soppressione di voti registrati; correlazione tra metadati e identità per violare il pseudo-anonimato; risalire all'identità dell'elettore tramite il $Cert_e$ ricevuto in fase di autenticazione (violazione C.5). Con l'integrazione blockchain: omissione selettiva di <code>submitBallotAnchor</code> (voti presenti off-chain ma invisibili on-chain, rilevabile confrontando il conteggio degli ancoraggi con N); omissione di <code>issueToken</code> per emettere token non ancorati on-chain, rilevabili in audit.
Gestore disonesto dell'AS	Confidenzialità (C.1, C.2), Integrità (I.4)	Decifratura di voti individuali per violare la segretezza; alterazione del risultato finale durante il conteggio.
Autorità di Autenticazione disonesta	Integrità (I.2, I.3, I.6), Confidenzialità (C.2), Efficienza (E.1)	Rilascio di credenziali a soggetti non aventi diritto o credenziali multiple; tracciamento degli elettori per dedurre preferenze; denial-of-service selettivo che impedisce a specifici elettori legittimi di votare in tempi ragionevoli.
Avversario esterno	Confidenzialità (C.4, C.5), Integrità (I.5), Efficienza	Intercettazione di credenziali e voti in transito; attacchi DoS contro le infrastrutture del sistema.
Elettore disonesto e coercitore	Integrità (I.3, I.5), Confidenzialità (C.6), Trasparenza (T.3)	Voto multiplo; vendita del voto con prova della preferenza; coercizione durante o dopo il voto.
Avversario passivo	Confidenzialità (C.2, C.4)	Correlazione di metadati di rete con l'identità dell'elettore, violando il pseudo-anonimato senza alterare alcun dato.

Analisi delle minacce

La tabella evidenzia come ogni avversario abbia un “profilo di minaccia” caratteristico. Notare che:

- La **Confidenzialità (C.2)** è la proprietà più minacciata, coinvolta da quattro avversari su sei.

- Gli **avversari interni** (gestori AA, AR, AS) hanno il potenziale di impatto maggiore grazie ai loro privilegi.
- L'**avversario passivo** è il meno potente ma anche il più difficile da rilevare, poiché non altera alcun dato.

Questa analisi guida la progettazione delle contromisure nel WP2, dove ogni proprietà minacciata viene protetta da specifici meccanismi crittografici.

6 Completezza

Le seguenti condizioni, prese congiuntamente, definiscono il comportamento atteso del sistema nel caso in cui tutti gli attori operino onestamente. La violazione di una singola condizione non implica necessariamente il fallimento dell'intera elezione, ma può compromettere specifiche proprietà identificate nella Sezione 4.

Cond.1: Se l'elettore è in possesso di una credenziale di voto valida rilasciata dall'Autorità di Autenticazione, può esprimere la propria preferenza tramite l'interfaccia di voto.

Cond.2: Se l'elettore esprime il proprio voto, l'Autorità di Raccolta registra correttamente il voto cifrato e rilascia un codice di ricevuta che permette all'elettore la successiva verifica.

Cond.3: Se la votazione termina regolarmente, l'Autorità di Scrutinio decifra i voti raccolti, calcola il risultato e lo pubblica insieme alle prove crittografiche necessarie.

Cond.4: Se l'elettore utilizza il codice di ricevuta ricevuto, può verificare che il proprio voto sia presente nel registro dei voti e sia stato conteggiato nel risultato finale.

Cond.5: Chiunque acceda ai dati pubblicati può controllare la correttezza del conteggio.

Relazione con le proprietà

Le condizioni di completezza sono le garanzie positive del sistema: descrivono *cosa deve accadere* quando tutto funziona correttamente. Le proprietà di sicurezza della Sezione 4, invece, descrivono *cosa non deve accadere* in presenza di avversari. Le due prospettive sono complementari e insieme definiscono lo spazio dei requisiti completo.

7 Relazioni di Fiducia

L'architettura proposta si fonda sui seguenti presupposti minimi di fiducia. Ogni assunzione è esplicitamente dichiarata per garantire la trasparenza richiesta dalla proprietà T.4.

7.1 Autorità di Autenticazione

L'Autorità di Autenticazione è considerata **fidata per l'identificazione corretta degli elettori** e per il rilascio di una sola credenziale per avente diritto. Tuttavia, **non è fidata per la segretezza del voto**: l'AA

conosce l'identità dell'elettore ma non deve mai avere accesso alla preferenza espressa.

7.2 Decifratura del Voto

Si assume che **nessuna singola entità detenga il controllo esclusivo sulla chiave di decifratura**, adottando un approccio di *threshold decryption*. In alternativa, l'Autorità di Scrutinio è fidata per la segretezza, ma l'integrità del conteggio deve essere universalmente verificabile.

7.3 Urna Elettronica

L'Urna Elettronica (gestita dall'AR) è considerata **fidata per la corretta registrazione dei voti**, che sono però cifrati; **non è quindi fidata per la loro segretezza**. Questa separazione garantisce che anche un AR compromessa non possa decifrare i contenuti dei voti.

7.4 Ente Certificatore Legale

L'Ente Certificatore Legale è considerato **fidato per l'attestazione temporale dell'apertura e della chiusura delle urne**, in quanto terza parte nominata da un organo istituzionale indipendente.

Importante – Impatto della compromissione dell'EC

La compromissione dell'EC invaliderebbe il valore legale del processo elettorale ma **non le proprietà crittografiche del protocollo**, che rimangono garantite indipendentemente dal suo operato. Questo è un esempio di *separazione delle garanzie*: le proprietà legali e quelle crittografiche sono indipendenti.

Table 3: Sintesi delle relazioni di fiducia

Entità	Affidabile per	Non affidabile per
Autorità di Autenticazione	Identificazione corretta, rilascio credenziali	Segretezza del voto
Autorità di Raccolta	Registrazione corretta dei voti	Decifratura, segretezza
Autorità di Scrutinio	Calcolo corretto del risultato	Associazione voto-identità (garantita solo se il numero di garanti colludenti è strettamente inferiore alla soglia t ; con $t = 2$ su $n = 3$, il sistema è sicuro finché almeno 2 garanti su 3 agiscono onestamente)
Ente Certificatore Legale	Attestazione temporale legale	Proprietà crittografiche
Ethereum / VotingContract	Immutabilità e disponibilità dei dati pubblicati on-chain; controllo anti-doppio-voto autonomo	Correttezza dei dati inseriti (il contratto registra ciò che le autorità scrivono: la blockchain garantisce che i dati non siano alterati <i>dopo</i> la scrittura, non che siano corretti <i>al momento</i> della scrittura)

Analisi delle assunzioni di fiducia

Il modello di fiducia è deliberatamente minimale. L'adozione dello schema *threshold* per la decifratura (WP2, Sezione 10.4) riduce ulteriormente il trust richiesto nell'AS. L'integrazione di Ethereum riduce il trust richiesto verso AR e AS per la *pubblicazione* dei dati: immutabilità e disponibilità sono garantite dalla rete, non dall'onestà delle singole autorità. Questo approccio, noto come "trust minimization", è un principio guida della progettazione del protocollo crittografico sviluppato nel WP2.

I rischi residui derivanti dalla violazione delle assunzioni di fiducia dichiarate, con riferimento alle proprietà minacciate identificate nella Tabella 2, esulano dagli obiettivi del presente documento e sono indicati come sviluppo futuro.

WORK PACKAGE 2 — Protocollo Crittografico

8 Introduzione WP2

Di seguito viene descritto il protocollo crittografico progettato per la conduzione sicura di un'elezione elettronica. Il WP1 ha definito il modello di sicurezza e le proprietà richieste; coerentemente con quanto stabilito, il WP2 sviluppa gli strumenti crittografici concreti per realizzarle.

9 Scelte Crittografiche Principali

Il protocollo si fonda sui seguenti strumenti crittografici, trattati nel corso e selezionati per garantire le proprietà di sicurezza individuate nel WP1.

Table 4: Strumenti crittografici e proprietà garantite

Strumento	Utilizzo nel protocollo e proprietà garantite
RSA / FDH	Autenticazione elettori, firma di voto e risultato. <i>Proprietà:</i> autenticità, non ripudio.
PKI / X.509	Distribuzione chiavi pubbliche delle autorità. <i>Proprietà:</i> fiducia nella chiave pubblica.
RSA-OAEP	Cifratura del voto verso l'Autorità di Scrutinio. <i>Proprietà:</i> confidenzialità (CPA-secure).
SHA-256	Integrità messaggi, receipt, hash di certificati, Merkle tree on-chain e commitment al voto tramite schema salted-hash $SHA-256(v r_{commit})$. <i>Proprietà:</i> integrità, collision resistance, hiding e binding sotto assunzione di resistenza alle collisioni.
Shamir threshold	Distribuzione chiave di decifratura AS. <i>Proprietà:</i> resistenza a singolo punto di fallimento.
RSA-OAEP + RSA-FDH (trasmissione share)	Protezione end-to-end delle share Shamir nella trasmissione da ciascun garante G_j ad AS. La share è cifrata con pk_{AS}^{recv} (confidenzialità) e firmata con sk_{G_j} (autenticità). Il canale di trasporto sottostante non deve essere fidato. <i>Proprietà:</i> confidenzialità (CPA-secure), autenticità, non ripudio, non-replay.
Nonce / Timestamp	Prevenzione replay, freschezza messaggi. <i>Proprietà:</i> unicità, non-replay.
Ethereum / Smart Contract	Registro pubblico immutabile per attestazioni, commitment, voti cifrati (C_{voto}), BB e controllo anti-doppio-voto. La pubblicazione on-chain di C_{voto} consente ad AS di recuperare i voti cifrati direttamente dalla chain. <i>Proprietà:</i> immutabilità, trasparenza, decentralizzazione (T.2, T.4, I.1, I.3).

Analisi della scelta crittografica

La combinazione di RSA-FDH per le firme e RSA-OAEP per la cifratura permette di separare chiaramente le garanzie: la firma garantisce *autenticità e non ripudio*, mentre la cifratura garantisce *confidenzialità*. L'uso di OAEP (Optimal Asymmetric Encryption Padding) è essenziale per ottenere la CPA-SECURITY e la non-malleabilità, proprietà che il semplice RSA testuale non possiede.

L'integrazione di Ethereum come livello di pubblicazione e registro aggiunge una garanzia ortogonale: l'immutabilità dei dati pubblicati non dipende più dalla correttezza o dalla disponibilità di un singolo server istituzionale, ma è garantita dal consenso distribuito della rete. Gli smart contract sono verificabili da chiunque tramite un qualsiasi nodo o block explorer. Le operazioni on-chain sono eseguite

dalle autorità (AR, AS, EC) tramite account Ethereum dedicati le cui chiavi pubbliche sono certificate dalla stessa PKI del sistema (sezione 10).

10 Generazione e Distribuzione delle Chiavi

Prima dell'apertura delle urne viene eseguita una fase di *setup* durante la quale vengono generate e distribuite tutte le chiavi del sistema. Tutte le chiavi pubbliche sono distribuite tramite certificati X.509 firmati da una Certification Authority istituzionale, il cui certificato radice è noto a tutti i partecipanti. Questo meccanismo previene attacchi *Man-in-the-Middle* durante la distribuzione delle chiavi pubbliche.

10.1 Flusso Comune di Distribuzione delle Chiavi

Il flusso di distribuzione è uniforme per tutte le autorità del sistema e si articola nei seguenti passi, eseguiti separatamente da ciascuna autorità prima dell'avvio del protocollo:

1. **Generazione locale:** l'autorità genera autonomamente la propria coppia di chiavi RSA (pk, sk) in un ambiente sicuro e isolato. La chiave privata sk **non lascia mai** questo ambiente.
2. **Richiesta di certificazione:** l'autorità invia a CA la propria chiave pubblica pk insieme al proprio identificativo ID , previa autenticazione istituzionale fuori banda (ad esempio tramite le credenziali).
3. **Emissione del certificato:** CA verifica l'identità del richiedente e, se conforme, firma il certificato:

$$Cert = \text{Sign}_{sk_{CA}}(pk \parallel ID \parallel \text{validità}),$$

e lo restituisce all'autorità richiedente.

4. **Pubblicazione:** l'autorità pubblica $Cert$ sul portale istituzionale. Da questo momento chiunque può ottenere la chiave pubblica autentica verificando la firma di CA tramite il certificato radice $Cert_{CA}$, noto a tutti i partecipanti.

I paragrafi seguenti descrivono le specificità di ogni autorità, che si aggiungono al flusso comune appena descritto.

10.2 Chiavi dell'Autorità di Autenticazione (AA)

- Genera una coppia di chiavi RSA: (pk_{AA}, sk_{AA}) .
- Ottiene da CA il certificato:

$$Cert_{AA} = \text{Sign}_{sk_{CA}}(pk_{AA} \parallel ID_{AA} \parallel \text{validità}).$$

- Pubblica $Cert_{AA}$ sul portale istituzionale; tutti gli elettori possono verificare l'autenticità di pk_{AA} tramite la firma della CA.
- sk_{AA} è mantenuta in modo esclusivo dall'AA e **non è mai condivisa** con altre autorità.

10.3 Chiavi dell'Autorità di Raccolta (AR)

- Genera una coppia di chiavi RSA: (pk_{AR}, sk_{AR}) .
- Ottiene da CA il certificato $Cert_{AR}$.
- sk_{AR} è utilizzata esclusivamente per autenticare le ricevute rilasciate agli elettori; **non è coinvolta** nella decifratura dei voti.

10.4 Chiavi dell'Autorità di Scrutinio (AS) – Schema Threshold

La chiave di decifratura dell'Autorità di Scrutinio è gestita con uno schema (t, n) -threshold di Shamir, con parametri $n = 3$ e $t = 2$, in modo che nessun singolo responsabile possa decifrare individualmente i voti.

- L'AS genera una coppia RSA (pk_{AS}, sk_{AS}) .
- sk_{AS} viene suddivisa in $n = 3$ share s_1, s_2, s_3 tramite lo schema di Shamir.
- Il polinomio e sk_{AS} vengono **distrutti** dopo la distribuzione delle condivisioni.
- Per decifrare è necessario che almeno $t = 2$ garanti conferiscano la propria condivisione, ricostruendo sk_{AS} mediante **interpolazione di Lagrange** dove $S \subseteq \{1, 2, 3\}$ con $|S| = 2$.
- pk_{AS} è pubblicata con certificato $Cert_{AS}$ firmato da CA.
- Poiché sk_{AS} (la chiave di decifratura) viene distrutta dopo la distribuzione delle share e non è disponibile fino alla fase di scrutinio, l'AS genera e mantiene una coppia di chiavi RSA **di firma** separata $(pk_{AS}^{sign}, sk_{AS}^{sign})$, certificata da CA tramite $Cert_{AS}^{sign}$. Questa chiave non viene mai suddivisa né distribuita ed è utilizzata esclusivamente per autenticare le comunicazioni dell'AS.
- L'AS genera inoltre una coppia di chiavi RSA dedicata alla **ricezione cifrata delle share**:

$$(pk_{AS}^{recv}, sk_{AS}^{recv}),$$

certificata da CA tramite $Cert_{AS}^{recv}$. Questa chiave non è mai suddivisa né distribuita; il suo unico scopo è decifrare i pacchetti cifrati inviati dai garanti nella fase di scrutinio (Sezione 16), eliminando la dipendenza da un canale di rete fidato.

10.5 Chiavi dei Garanti

Ciascun garante G_j ($j = 1, 2, 3$) partecipa alla fase di setup seguendo il flusso comune descritto nella Sezione 10.1:

- Genera una coppia di chiavi RSA (pk_{G_j}, sk_{G_j}) in un ambiente locale sicuro.
- Ottiene da CA il certificato:

$$Cert_{G_j} = \text{Sign}_{sk_{CA}}(pk_{G_j} \parallel ID_{G_j} \parallel \text{validità}).$$

- Pubblica $Cert_{G_j}$ sul portale istituzionale; AS può così verificare l'autenticità delle share ricevute.
- sk_{G_j} è mantenuta localmente dal garante e **non è mai trasmessa** ad alcuna autorità.

10.6 Chiavi dell'Elettore

- Ogni elettore genera una coppia RSA (pk_e, sk_e) al momento della registrazione.
- L'AA certifica la coppia $(ID_{\text{elettore}}, pk_e)$ previa verifica dell'identità, rilasciando:

$$Cert_e = \text{Sign}_{sk_{AA}}(pk_e \parallel ID_{\text{elettore}}).$$

- sk_e è mantenuta localmente dall'elettore e **non è mai trasmessa** ad alcuna autorità.

11 Registrazione degli Elettori

La fase di registrazione avviene prima dell'apertura delle urne e si apre con un'azione autonoma da parte dell'**Elettore**, il quale agisce nel proprio ambiente locale per garantire la sicurezza del sistema.

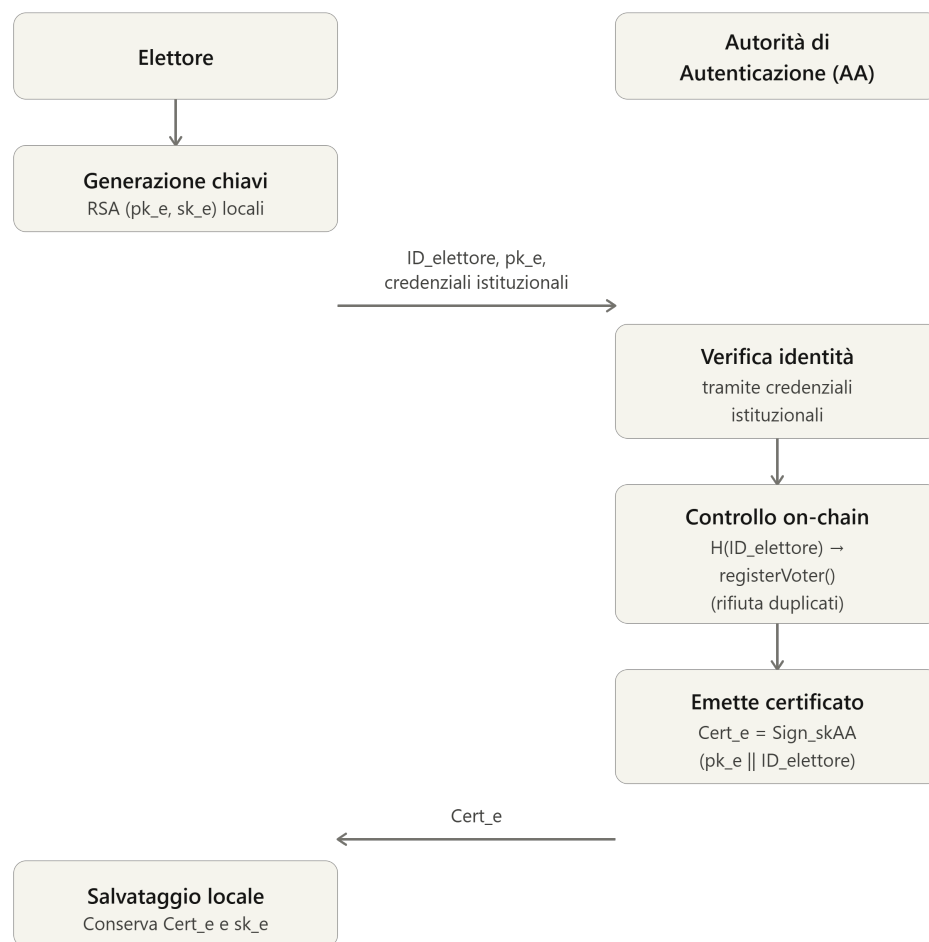


Figure 1: Diagramma del flusso di registrazione degli elettori

11.1 Generazione delle Chiavi Locali

L'Elettore genera una coppia di chiavi crittografiche RSA:

- pk_e : chiave pubblica, destinata a essere condivisa;

- sk_e : chiave privata, che rimane segreta e sotto il suo esclusivo controllo.

11.2 Richiesta di Identificazione

L'Elettore si rivolge all'**Autorità di Autenticazione** inviando un pacchetto contenente:

- Identificativo univoco ID_{elettore} ;
- Chiave pubblica pk_e appena generata;
- Credenziali istituzionali.

Questo passaggio lega l'identità digitale tecnica alla reale identità burocratica dell'utente.

Istanziamento pratico in un contesto reale

In un sistema di voto nazionale italiano, ID_{elettore} potrebbe corrispondere al **Codice Fiscale** (CF) del cittadino: identificativo stabile, univoco e già adottato dalla pubblica amministrazione. Poiché il protocollo pubblica on-chain esclusivamente $H(ID_{\text{elettore}})$, il CF grezzo non è mai esposto sulla blockchain. Le **credenziali istituzionali** potrebbero essere realizzate tramite la **Carta d'Identità Elettronica** (CIE) con PIN.

11.3 Validazione e Emissione del Certificato

L'Autorità di Autenticazione esegue un duplice controllo:

1. **Verifica dell'identità:** accerta che le credenziali istituzionali siano valide e appartengano effettivamente al richiedente.
2. **Registrazione anti-duplicato (on-chain):** calcola $\text{voterIdHash} = H(ID_{\text{elettore}})$ e invoca

`VotingContract.registerVoter( $H(ID_{\text{elettore}})$ )`.

Il contratto verifica che $H(ID_{\text{elettore}})$ non sia già presente nel mapping `registeredVoters`: se è già registrato, la transazione è rifiutata e AA non emette un nuovo certificato. Se il controllo passa, $H(ID_{\text{elettore}})$ viene registrato in modo immutabile. Questo impedisce che lo stesso elettore ottenga più certificati generando coppie di chiavi diverse, poiché il controllo dipende dall'identità stabile dell'elettore e non dalla chiave pubblica o dal certificato prodotto.

Superata la verifica, l'AA produce il certificato:

$$Cert_e = \text{Sign}_{sk_{AA}}(pk_e \parallel ID_{\text{elettore}}). \quad (1)$$

11.4 Conclusione della Procedura

Il certificato viene trasmesso all'Elettore, che conserva in modo sicuro sia $Cert_e$ che sk_e . Da questo momento l'utente è ufficialmente abilitato a interagire con il sistema di voto in modo autenticato e protetto.

12 Apertura delle Urne

Prima dell'apertura ufficiale delle urne, l'Autorità di Autenticazione chiude la fase di registrazione e congela la lista degli aventi diritto.

12.1 Richiesta di Attestazione

L'AA trasmette all'**Ente Certificatore** (EC) la seguente richiesta:

$$\text{"APERTURA"} \parallel ts_{\text{apertura}} \quad (2)$$

L'EC emette l'attestazione di apertura:

$$Att_{\text{apertura}} = \text{Sign}_{sk_{EC}}(\text{"APERTURA"} \parallel ts_{\text{apertura}}). \quad (3)$$

Separazione tra garanzia tecnica e garanzia legale

L'attestazione non include la lista degli aventi diritto: l'immutabilità di quest'ultima è già garantita per costruzione dallo smart contract. Il mapping `registeredVoters` on-chain, reso immutabile dalla transizione di stato `CREATED` $\rightarrow$ `OPEN`, costituisce il registro tecnico degli aventi diritto; chiunque può verificarne il contenuto interrogando direttamente il contratto o ispezionando gli eventi `VoterRegistered` su un block explorer pubblico. Il ruolo di EC è distinto: conferire **valore legale certificato** al momento di apertura delle urne, in conformità con la proprietà **T.5**. Le due responsabilità — tecnica e legale — rimangono separate e assegnate alle entità corrette, in linea con il principio dichiarato nella Sezione 7.

EC invia quindi una transazione allo smart contract di elezione su Ethereum che registra in modo immutabile:

$$\text{VotingContract.openElection}(ts_{\text{apertura}}, Att_{\text{apertura}}).$$

La funzione è protetta dal modificatore `onlyEC`: solo l'account Ethereum dell'EC ($addr_{EC}$) può invocarla. Una volta eseguita, il contratto emette l'evento `ElectionOpened` e blocca qualsiasi ulteriore chiamata a `openElection`, rendendo il momento di apertura **immutabile on-chain**. Chiunque può verificare il dato leggendo lo stato del contratto o ispezionando l'evento su un block explorer pubblico, senza dover fidare del portale istituzionale.

13 Autenticazione e Rilascio del Token di Sessione

Il processo ha inizio con una nuova azione dell'**Elettore**, volta a dimostrare la propria identità e la freschezza della richiesta.

13.1 Generazione della Richiesta Autenticata

Per evitare attacchi di tipo *replay*, l'Elettore genera:

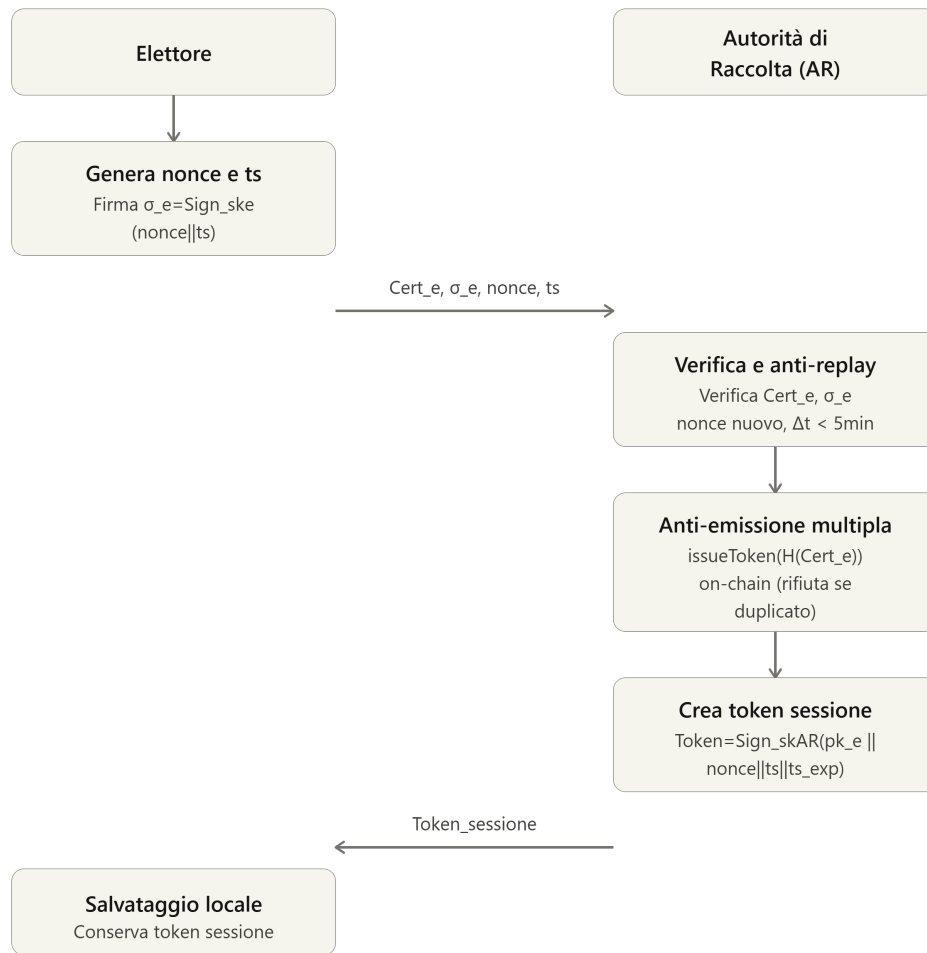


Figure 2: Diagramma del flusso di autenticazione e rilascio del token di sessione

- $nonce \in_R \{0, 1\}^{128}$: numero casuale *usa-e-getta*;
- ts : timestamp corrente.

Subito dopo, appone la firma digitale:

$$\sigma_e = \text{Sign}_{sk_e}(\text{nonce} || ts). \quad (4)$$

13.2 Invio della Richiesta

L'Elettore invia all'**Autorità di Raccolta** il pacchetto:

$$(Cert_e, \sigma_e, nonce, ts).$$

13.3 Validazione e Controlli Anti-Frode

L'Autorità esegue quattro controlli critici:

1. **Verifica del certificato:** controlla che $Cert_e$ sia autentico usando pk_{AA} (da $Cert_{AA}$) ed estrae pk_e .
2. **Verifica della firma:** verifica che $\text{Vrfy}_{pk_e}(\text{nonce} || ts, \sigma_e) = 1$.

3. **Controllo anti-replay:** verifica che il nonce non sia già stato registrato e che

$$|ts - ts_{\text{corrente}}| < \Delta_{\text{max}}, \quad \Delta_{\text{max}} = 5 \text{ min.}$$

4. **Controllo anti-emissione-multipla (on-chain):** invoca

$$\text{VotingContract.issueToken}(H(Cert_e))$$

Il contratto verifica che $H(Cert_e)$ non sia già presente nel mapping `issuedTokens`: se è già registrato, la transazione è rifiutata e AR non emette alcun token. Se il controllo passa, $H(Cert_e)$ viene registrato in modo immutabile. Questo impedisce che lo stesso elettore ottenga più token validi presentando nonce e timestamp diversi a ogni nuova autenticazione.

13.4 Creazione del Token di Sessione

Superati tutti i controlli, AR estrae pk_e dal certificato verificato e genera il token di sessione:

$$Token_{\text{sessione}} = \text{Sign}_{sk_{AR}}(pk_e \parallel nonce \parallel ts \parallel ts_{\text{exp}}), \quad (5)$$

dove ts_{exp} è il timestamp di scadenza del token.

Proprietà del token di sessione

Il token garantisce che chi vota ne abbia effettivamente diritto (**I.2**) ed è monouso: una volta consumato in `usedTokens` on-chain, non può essere riutilizzato per un secondo voto (**I.3**). Il campo ts_{exp} , incluso nel dato firmato da AR, è crittograficamente non falsificabile: qualsiasi modifica invalida la firma di AR. La verifica che $ts_{\text{exp}} > ts_{\text{corrente}}$ è tuttavia delegata ad AR in fase off-chain durante la ricezione della scheda (Sezione 14); il contratto non esegue questo controllo autonomamente, affidandosi al modificatore di stato `inState(OPEN)` come fallback temporale di granularità più grossolana. Il mapping `issuedTokens` on-chain garantisce inoltre che per ciascun elettore venga emesso **al più un token** per l'intera durata dell'elezione (**I.3**): tentativi di autenticazione successivi sono rifiutati dal contratto prima ancora che AR possa generare un nuovo token.

14 Formato della Scheda di Voto e Operazioni Crittografiche

14.1 Struttura della Scheda di Voto Cifrata

Il voto espresso dall'elettore è un bit $v \in \{0, 1\}$ (0 = No, 1 = Sì). La scheda di voto è composta dai seguenti campi:

Table 5: Campi della scheda di voto cifrata

Campo	Descrizione e scopo
C_{voto}	Cifratura RSA-OAEP del voto v con fattore di randomizzazione $r_{\text{voto}} \in_R \{0, 1\}^{128}$. Garantisce CPA-security: due cifrature dello stesso voto producono cifrati diversi. AR pubblica C_{voto} on-chain nella fase di ancoraggio, consentendo ad AS di recuperarlo direttamente dalla blockchain senza canale off-chain.
$\text{Commit}_{\text{voto}}$	Commitment salted-hash al voto: $\text{SHA-256}(v \parallel r_{\text{commit}})$ con $r_{\text{commit}} \in_R \{0, 1\}^{256}$. Permette verifica individuale e universale a posteriori senza rivelare il voto durante la raccolta. La proprietà di <i>hiding</i> è garantita dalla resistenza alla preimmagine di SHA-256; la proprietà di <i>binding</i> è garantita dalla resistenza alle collisioni. Il fattore r_{commit} non viene trasmesso nella scheda: è conservato localmente dall'elettore e rivelato solo in fase di verifica individuale.
$\text{token}_{\text{sessione}}$	Token di sessione rilasciato da AR nella fase di autenticazione. Prova che l'elettore è autorizzato e contiene nel proprio payload firmato pk_e , consentendo ad AR di recuperare il riferimento alla chiave pubblica dell'elettore senza che Cert_e sia ritrasmesso nella scheda.
ts_{voto}	Timestamp del momento di espressione del voto.
$\text{nonce}_{\text{voto}}$	Nonce casuale per prevenire replay.
σ_{scheda}	Firma dell'elettore sull'intera scheda. Garantisce integrità e autenticità.

La scheda completa trasmessa ad AR è:

$$B = (C_{\text{voto}}, \text{Commit}_{\text{voto}}, \text{token}_{\text{sessione}}, ts_{\text{voto}}, \text{nonce}_{\text{voto}}, \sigma_{\text{scheda}}) \quad (6)$$

La firma dell'elettore è calcolata come:

$$\sigma_{\text{scheda}} = \text{Sign}_{sk_e} \left(H(C_{\text{voto}} \parallel \text{Commit}_{\text{voto}} \parallel \text{token}_{\text{sessione}} \parallel ts_{\text{voto}} \parallel \text{nonce}_{\text{voto}}) \right). \quad (7)$$

14.2 Operazioni Crittografiche – Dettaglio

14.2.1 Cifratura del Voto (Elettore)

1. Genera $r_{\text{voto}} \in_R \{0, 1\}^{128}$.

2. Calcola:

$$C_{\text{voto}} = \text{Enc}_{pk_{AS}}^{\text{OAEP}}(v \parallel r_{\text{voto}}). \quad (8)$$

La modalità OAEP garantisce CPA-security e non-malleabilità.

3. Genera $r_{\text{commit}} \in_R \{0, 1\}^{256}$.

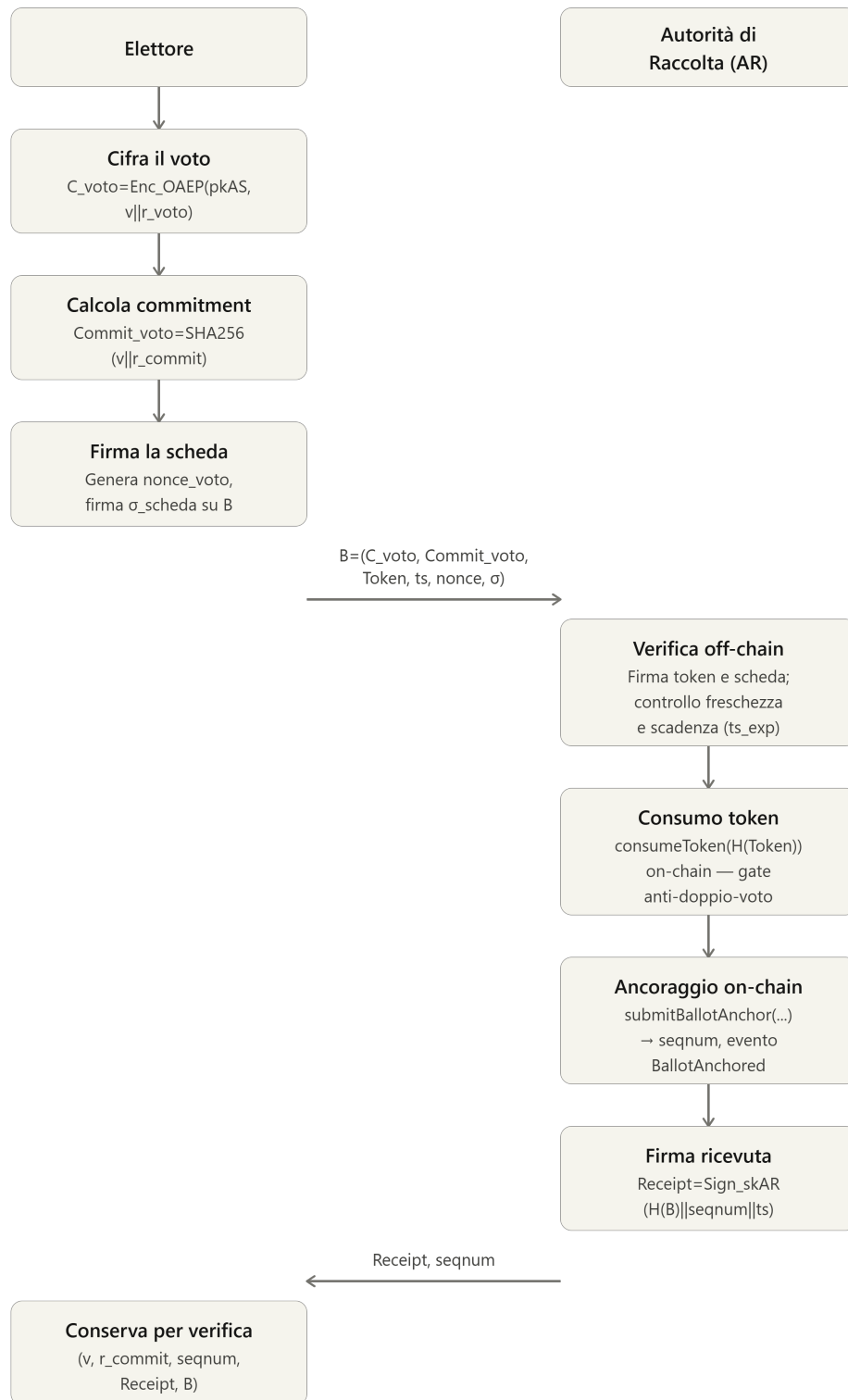


Figure 3: Diagramma del flusso di espressione e verifica della scheda di voto

4. Calcola il commitment:

$$Commit_{voto_i} = SHA-256(v_i \parallel r_{commit_i}). \quad (9)$$

SHA-256 garantisce la proprietà di *hiding*: AR non può ricavare v_i dal solo $Commit_{voto_i}$. La resistenza alle collisioni garantisce la proprietà di *binding*: non esiste alcuna coppia (v'_i, r') con $v'_i \neq v_i$ tale che $SHA-256(v'_i \parallel r') = SHA-256(v_i \parallel r_{commit_i})$.

5. Genera $nonce_{\text{voto}} \in_R \{0, 1\}^{128}$.
6. Calcola il digest e la firma secondo l'Equazione (7).

14.2.2 Verifica della Scheda (AR)

I controlli sono eseguiti in due fasi distinte: prima i controlli **off-chain** (veloci e reversibili), poi il **consumo del token on-chain** (lento e irreversibile). Questo ordine è essenziale: consumare il token prima di verificare la firma esporrebbe il sistema a un attacco DoS mirato, in cui un avversario in possesso di un token intercettato potrebbe inviare una scheda con firma invalida, consumando il token e impedendo all'elettore legittimo di votare.

1. Verifica firma elettore (off-chain):

- (a) Verifica la firma di AR sul token: $\text{Vrfy}_{pk_{AR}}(token_{\text{sessione}}) = 1$. Se non passa, la scheda è rifiutata senza alcuna operazione on-chain.
- (b) Estrae pk_e dal payload del token.
- (c) Ricalcola il digest:

$$d' = H(C_{\text{voto}} \parallel Commit_{\text{voto}} \parallel token_{\text{sessione}} \parallel ts_{\text{voto}} \parallel nonce_{\text{voto}}).$$

- (d) Verifica $\text{Vrfy}_{pk_e}(d', \sigma_{\text{scheda}}) = 1$. Se non passa, la scheda è rifiutata senza alcuna operazione on-chain.

2. Verifica freschezza e scadenza token (off-chain): AR controlla congiuntamente:

- (a) $|ts_{\text{voto}} - ts_{\text{corrente}}| < 5 \text{ min}$;
- (b) $nonce_{\text{voto}}$ non già registrato nel registro locale;
- (c) ts_{exp} (estratto dal payload del token verificato al passo precedente) $> ts_{\text{corrente}}$.

Se uno qualsiasi di questi controlli fallisce, la scheda è rifiutata senza alcuna operazione on-chain. Il controllo su ts_{exp} è essenziale: senza di esso, un token sottratto all'elettore potrebbe essere presentato validamente da un avversario anche ore dopo la sua emissione, fintanto che l'elezione è ancora in stato OPEN.

3. Consumo del token (gate finale on-chain): Solo dopo che tutti i controlli off-chain hanno avuto esito positivo, AR invoca

`VotingContract.consumeToken( $H(token_{\text{sessione}})$ )`: il contratto verifica atomicamente che l'hash non sia già nel mapping `usedTokens` on-chain (prevenzione doppio voto) e, se il controllo passa, lo registra in modo immutabile. Solo se la transazione è confermata AR procede; altrimenti, la scheda è rifiutata.

4. Registrazione: se tutti i controlli passano, AR pubblica on-chain l'ancoraggio invocando:

$$\text{VotingContract.submitBallotAnchor}(H(token_{\text{sessione}}), H(B), \\ Commit_{\text{voto}}, C_{\text{voto}}, ts_{\text{registrazione}}).$$

L'inclusione di C_{voto} on-chain consente ad AS di recuperare i voti cifrati direttamente dalla blockchain nella fase di scrutinio. Il contratto assegna internamente il numero progressivo $seq_{\text{num}} = \text{ballotAnchors.length}$ (prima del push), aggiunge la tupla $(seq_{\text{num}}, H(B), \text{Commit}_{\text{voto}}, C_{\text{voto}})$ all'array `ballotAnchors` ed emette l'evento `BallotAnchored(seq_num, H(B), Commit, ts)`. AR **non** passa seq_{num} come argomento: il valore è determinato deterministicamente dallo stato del contratto, impedendo che AR manipoli l'ordinamento degli ancoraggi.

Ordine delle operazioni: prima la transazione

Poiché seq_{num} è assegnato dal contratto e reso noto tramite l'evento `BallotAnchored`, AR **deve attendere la conferma della transazione** prima di poter firmare la ricevuta. Solo dopo aver letto seq_{num} dal log dell'evento confermato, AR produce:

$$Receipt = \text{Sign}_{sk_{AR}}(H(B) \parallel seq_{\text{num}} \parallel ts_{\text{registrazione}}), \quad (10)$$

dove seq_{num} è il valore emesso dall'evento on-chain. AR trasmette all'elettore sia *Receipt* sia seq_{num} : l'elettore li conserva localmente insieme a $(v, r_{\text{commit}}, B)$ per la verifica individuale. L'elettore può verificare autonomamente la correttezza di seq_{num} leggendo direttamente lo smart contract, senza dover fidarsi di AR.

15 Chiusura delle Urne

Al termine del periodo di votazione, l'Autorità di Raccolta esegue le seguenti operazioni.

15.1 Attestazione di Chiusura

L'AR segnala all'EC la conclusione della raccolta dei voti. Prima di invocare il contratto, EC emette l'attestazione di chiusura:

$$Att_{\text{chiusura}} = \text{Sign}_{sk_{EC}}("CHIUSURA" \parallel ts_{\text{chiusura}}). \quad (11)$$

Successivamente EC invoca

$$\text{VotingContract.closeElection}(ts_{\text{chiusura}}, Att_{\text{chiusura}})$$

sul contratto. Il contratto calcola autonomamente $MerkleRoot_{\text{urna}}$ dagli ancoraggi presenti in `ballotAnchors`, registra

$$scrutinyDeadline = ts_{\text{chiusura}} + \Delta_{\text{scrutinio}}$$

(con $\Delta_{\text{scrutinio}}$ parametro configurabile, tipicamente 72 ore), e transita nello stato `CLOSED`, bloccando qualsiasi ulteriore invocazione di `issueToken`, `consumeToken` e `submitBallotAnchor`.

16 Ricostruzione della Chiave e Decifratura

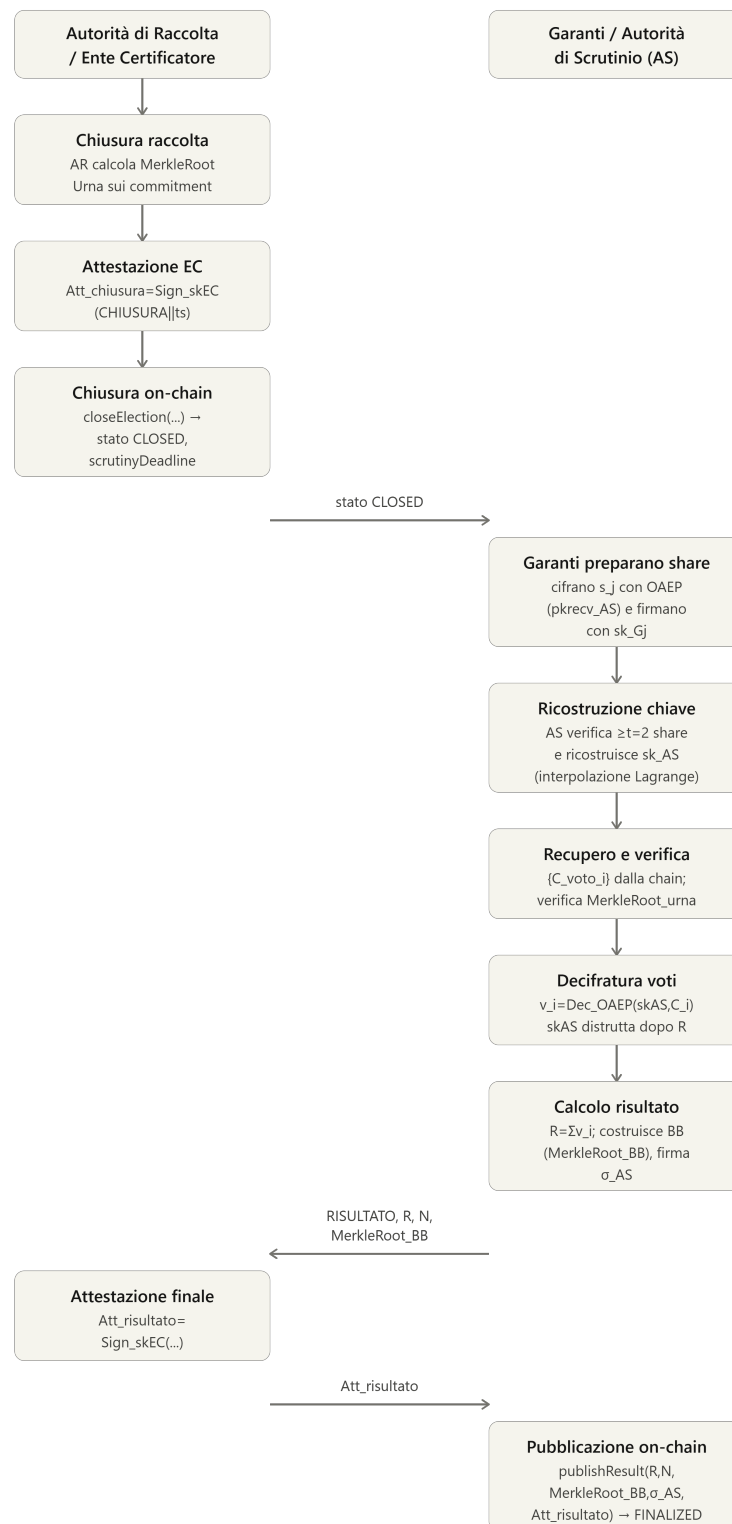


Figure 4: Diagramma del flusso di chiusura, ricostruzione della chiave, decifratura e pubblicazione del risultato

Al momento dello scrutinio, l'AS recupera direttamente dalla blockchain l'insieme dei voti cifrati $\{C_{voto_i}\}_{i=1}^N$, che AR ha pubblicato on-chain durante la fase di ancoraggio.

Successivamente si procede con la fase di decifratura, dove almeno $t = 2$ garanti (su $n = 3$) conferiscono le proprie condivisioni per ricostruire sk_{AS} .

16.1 Protocollo di Ricostruzione

1. Ogni garante G_j verifica che le urne siano ufficialmente chiuse interrogando direttamente lo smart contract e controllandone lo stato:

`VotingContract.state == State.CLOSED.`

Solo dopo questa verifica il garante è autorizzato a partecipare alla fase di ricostruzione.

2. Ciascun garante G_j protegge crittograficamente la propria share prima della trasmissione, eliminando la dipendenza da un canale di rete fidato. Il protocollo è il seguente:

- (a) Il garante genera $nonce_j \in_R \{0, 1\}^{128}$ e il timestamp ts_j .
- (b) Cifra la share con la chiave di ricezione di AS (certificata da CA, Sezione 10.4):

$$c_j = \text{Enc}_{pk_{AS}^{\text{recv}}}^{\text{OAEP}}(s_j \parallel ID_{G_j} \parallel nonce_j \parallel ts_j).$$

- (c) Firma il cifrato con la propria chiave privata:

$$\sigma_{G_j} = \text{Sign}_{sk_{G_j}}(c_j).$$

- (d) Invia il pacchetto $(Cert_{G_j}, c_j, \sigma_{G_j})$ ad AS tramite qualsiasi canale di rete, anche non fidato.

Per ciascun pacchetto ricevuto, AS esegue i seguenti controlli prima di accettare la share:

- (a) Verifica $Cert_{G_j}$ tramite pk_{CA} ed estrae pk_{G_j} .
- (b) Verifica $\text{Vrfy}_{pk_{G_j}}(c_j, \sigma_{G_j}) = 1$; se non passa, il pacchetto è scartato.
- (c) Decifra: $(s_j \parallel ID_{G_j} \parallel nonce_j \parallel ts_j) = \text{Dec}_{sk_{AS}^{\text{recv}}}^{\text{OAEP}}(c_j)$.
- (d) Verifica che ID_{G_j} corrisponda a un garante atteso, che $nonce_j$ non sia già stato registrato e che $|ts_j - ts_{\text{corrente}}| < \Delta_{\text{max}}$.

La cifratura con pk_{AS}^{recv} garantisce che solo AS possa leggere s_j (confidenzialità); la firma con sk_{G_j} garantisce che AS possa verificare l'identità del mittente senza dipendere dall'integrità del canale (autenticità). Un avversario che intercetti il pacchetto ottiene solo c_j , non decifrabile senza sk_{AS}^{recv} .

3. Raggiunta la soglia minima prevista dallo schema di Shamir (t, n) , l'AS ricostruisce la chiave privata sk_{AS} mediante interpolazione di Lagrange.
4. L'AS recupera dalla blockchain tutti gli ancoraggi delle schede e, per ogni elettore i , estrae il corrispondente voto cifrato:

$$C_{\text{voto}_i} = \text{VotingContract.ballotAnchors}[i].C_{\text{voto}}.$$

5. L'AS verifica l'integrità dell'urna ricalcolando $\text{MerkleRoot}_{\text{urna}}$ a partire dagli ancoraggi registrati on-chain e confrontandola con il valore memorizzato nello smart contract.

6. Verificata l'integrità dei dati, l'AS decifra ciascun voto:

$$v_i = \text{Dec}_{sk_{AS}}^{\text{OAEP}}(C_{\text{voto}_i}). \quad (12)$$

Distruzione della chiave di scrutinio

Al termine dello scrutinio e della pubblicazione dei risultati, la chiave privata sk_{AS} viene distrutta. In questo modo non è più possibile decifrare nuovamente i voti individuali, limitando il rischio di compromissioni future e rafforzando la riservatezza delle preferenze espresse dagli elettori.

16.2 Calcolo e Pubblicazione del Risultato

1. L'AS calcola il risultato:

$$R = \sum_{i=1}^N v_i. \quad (13)$$

2. L'AS costruisce il **Bulletin Board** (BB) iniziando dalla costruzione del Merkle tree con foglie:

$$H(v_i \parallel \text{Commit}_{\text{voto}_i}), \quad (14)$$

ottenendo MerkleRoot_{BB} .

3. L'AS firma il Bulletin Board con la propria chiave di firma sk_{AS}^{sign} :

$$\sigma_{AS} = \text{Sign}_{sk_{AS}^{\text{sign}}}(R \parallel N \parallel \text{MerkleRoot}_{BB}). \quad (15)$$

La chiave di decifratura sk_{AS} viene **distrutta immediatamente** dopo il calcolo del risultato R . La firma del BB utilizza correttamente sk_{AS}^{sign} (la chiave di firma dedicata, mai distribuita né distrutta), consentendo a chiunque di verificare σ_{AS} tramite pk_{AS}^{sign} anche in futuro.

Il BB viene quindi popolato e pubblicato con i seguenti contenuti:

- La lista completa dei voti decifrati $\{v_i\}_{i=1}^N$ **senza associazione all'identità**;
- La lista delle coppie $\{(\text{Commit}_{\text{voto}_i}, v_i)\}$ per verifica universale (la verifica di apertura dei commitment è delegata agli elettori che rivelano r_{commit} nella fase di verifica individuale);
- Il risultato finale R e il totale N ;
- MerkleRoot_{BB} e la firma σ_{AS} .

4. L'AS trasmette all'EC:

$$\text{"RISULTATO"} \parallel R \parallel N \parallel ts_{\text{risultato}} \parallel \text{MerkleRoot}_{BB}, \quad (16)$$

e riceve l'attestazione:

$$\text{Att}_{\text{risultato}} = \text{Sign}_{sk_{EC}}(\text{"RISULTATO"} \parallel R \parallel N \parallel ts_{\text{risultato}} \parallel \text{MerkleRoot}_{BB}). \quad (17)$$

L'AS aggiunge $Att_{risultato}$ al BB pubblicato; da questo momento il risultato è ufficialmente attestato e verificabile da chiunque tramite pk_{EC} .

Pubblicazione finale on-chain del Bulletin Board

Ottenuta $Att_{risultato}$, l'AS invoca sullo smart contract:

$$\text{VotingContract.publishResult}(R, N, \text{MerkleRoot}_{BB}, \sigma_{AS}, Att_{risultato}).$$

La funzione è protetta da `onlyAS` e verifica che lo stato del contratto sia `CLOSED` (elezione chiusa). Essa registra in modo immutabile il risultato finale R , il numero totale di voti N , la radice del Bulletin Board MerkleRoot_{BB} , la firma σ_{AS} e l'attestazione legale $Att_{risultato}$. Il contratto emette l'evento `ResultPublished` e transita nello stato `FINALIZED`, bloccando qualsiasi ulteriore modifica. Da questo momento chiunque, senza alcuna credenziale, può leggere R direttamente dalla chain e verificare la firma di AS e l'attestazione dell'EC, soddisfacendo in modo robusto la proprietà **T.2** (verificabilità universale).

17 Meccanismi di Verifica

Il protocollo supporta due livelli di verifica: *individuale* (per l'elettore) e *universale* (per qualsiasi osservatore esterno).

17.1 Verifica Individuale (Elettore)

Ogni elettore che abbia conservato localmente $(v, r_{\text{commit}}, seq_{\text{num}}, Receipt, B)$ può eseguire tre controlli distinti.

Primo controllo — Il commitment è ancorato on-chain? L'elettore legge direttamente dallo smart contract il campo `commit` dell'ancora registrata all'indice seq_{num} :

$$\text{VotingContract.ballotAnchors}[seq_{\text{num}}].\text{commit} \stackrel{?}{=} \text{Commit}_{\text{voto}}.$$

Dove $\text{Commit}_{\text{voto}} = \text{SHA-256}(v \parallel r_{\text{commit}})$ è il valore calcolato localmente dall'elettore al momento del voto. Il confronto avviene senza rivelare né v né r_{commit} : il commitment è opaco per chiunque non conosca entrambi i valori segreti. Questo controllo attesta che AR ha ancorato on-chain esattamente il commitment prodotto dall'elettore, senza alterazioni. L'apertura del commitment e la verifica del conteggio richiedono la pubblicazione del BB e sono eseguite nel terzo controllo. L'elettore può eseguire questo primo controllo **prima della chiusura delle urne**, senza attendere lo scrutinio.

Secondo controllo — AR ha firmato correttamente la ricevuta? L'elettore verifica la firma di AR sulla ricevuta, che lega insieme $H(B)$, seq_{num} e il timestamp di registrazione:

$$\text{Vrfy}_{pk_{AR}}(H(B) \parallel seq_{\text{num}} \parallel ts, Receipt) \stackrel{?}{=} 1. \quad (18)$$

Un esito positivo attesta che AR ha ricevuto esattamente quella scheda e le ha assegnato quel numero progressivo in modo non ripudiabile. Anche questo controllo è eseguibile **prima della chiusura delle urne**.

Terzo controllo — Il voto è stato conteggiato correttamente? Solo dopo la pubblicazione del Bulletin Board da parte di AS, l'elettore rivela localmente r_{commit} ed esegue due sotto-verifiche distinte:

1. **Apertura del commitment** (`verify_commit`): verifica che il commitment ancorato on-chain (confermato nel primo controllo) sia coerente con il voto espresso:

$$\text{VotingContract.ballotAnchors}[seq_{\text{num}}].\text{commit} \stackrel{?}{=} \text{SHA-256}(v || r_{\text{commit}}). \quad (19)$$

Grazie alla proprietà *binding* di SHA-256, AS non può pubblicare nel BB un voto $v' \neq v$ che sia coerente con il commitment on-chain senza violare la resistenza alle collisioni.

2. **Corrispondenza con il Bulletin Board**: verifica che il voto in chiaro pubblicato da AS nel BB coincida con quello espresso:

$$\text{BB.voto}[seq_{\text{num}}] \stackrel{?}{=} v. \quad (20)$$

Solo se entrambi i controlli sono soddisfatti l'elettore ha la certezza crittografica che il proprio voto sia stato decifrato e conteggiato correttamente.

Garanzia della verifica individuale

I primi due controlli attestano che AR ha memorizzato e ancorato correttamente la scheda inviata; sono eseguibili in qualsiasi momento dopo il voto. Il terzo controllo attesta la correttezza del conteggio da parte di AS ed è disponibile soltanto dopo la pubblicazione del BB. La combinazione dei tre controlli garantisce la proprietà **T.3** (verificabilità individuale) definita nel WP1, ed è accessibile a utenti non esperti se supportata da uno strumento dedicato (requisito **E.2**).

Limite dichiarato

La verifica individuale si basa sulla correttezza del BB pubblicato da AS. Se AS pubblicasse un BB alterato in modo *globalmente* coerente, un singolo elettore non potrebbe rilevarlo da solo: è la **verifica universale** a fornire questa garanzia, rendendo i due livelli complementari e necessari l'uno all'altro.

17.2 Verifica Universale

Chiunque, senza disporre di alcuna chiave privata, può verificare il risultato finale eseguendo i seguenti controlli. Tutti i dati necessari sono disponibili **direttamente on-chain**.

1. **Verifica della coerenza dei commitment AR ↔ AS**. Durante la raccolta AR pubblica on-chain, per ogni scheda, il commitment $\text{Commit}_i^{\text{AR}}$; dopo lo scrutinio AS pubblica nel BB il commitment aperto $\text{Commit}_i^{\text{AS}}$ insieme al voto in chiaro v_i . Il verificatore confronta i due valori:

$$\text{Commit}_i^{\text{AR}} = \text{VotingContract.ballotAnchors}[i].\text{commit}$$

$$\text{Commit}_i^{\text{AS}} = \text{BB.commitment}[i]$$

$$Commit_i^{AR} \stackrel{?}{=} Commit_i^{AS} \quad \forall i = 1, \dots, N. \quad (21)$$

Grazie alla proprietà *binding* di SHA-256, se i due valori coincidono AS ha necessariamente pubblicato il voto corretto: non esiste alcuna coppia (v'_i, r') con $v'_i \neq v_i$ tale che $SHA-256(v'_i \| r') = SHA-256(v_i \| r_{commit_i})$ senza violare la resistenza alle collisioni di SHA-256.

2. **Verifica del conteggio.** Il verificatore ricalcola il risultato a partire dalla lista in chiaro pubblicata nel BB e lo confronta con il valore registrato on-chain:

$$R \stackrel{?}{=} \sum_{i=1}^N v_i. \quad (22)$$

Il valore R è leggibile direttamente tramite `VotingContract.finalResult()`.

3. **Verifica dell'integrità del Bulletin Board.** Il verificatore controlla la firma di AS sul BB:

$$Vrfy_{pk_{AS}^{sign}}(R \| N \| MerkleRoot_{BB}, \sigma_{AS}) \stackrel{?}{=} 1.$$

Un esito positivo garantisce che il BB non sia stato alterato dopo la pubblicazione e che AS non possa disconoscerne il contenuto.

4. **Verifica dell'attestazione legale.** Il verificatore controlla l'attestazione dell'Ente Certificatore:

$$Vrfy_{pk_{EC}}("RISULTATO" \| R \| N \| ts_{risultato} \| MerkleRoot_{BB}, Att_{risultato}) \stackrel{?}{=} 1.$$

L'attestazione è recuperabile on-chain tramite `VotingContract.attRisultato()`.

17.3 Revocabilità e Sostituzione del Voto

17.3.1 Motivazione della Scelta

Come dichiarato nella proprietà **C.7** del WP1, il sistema **non supporta** la revoca né la sostituzione del voto dopo la registrazione. Questa è una scelta progettuale deliberata:

- Un meccanismo di revoca consentirebbe a un coercitore di imporre all'elettore di annullare il voto espresso e riesprimerlo sotto controllo, vanificando la resistenza alla coercizione (**C.6**).
- Un meccanismo di sostituzione faciliterebbe la vendita del voto: l'elettore potrebbe essere pagato per votare in un certo modo e dimostrarlo.

Table 6: Analisi costi-benefici della non-revocabilità

Aspetto	Impatto
Sicurezza (coercizione)	Positivo: nessun meccanismo di coercizione possibile post-voto
Sicurezza (compravendita)	Positivo: impossibile dimostrare come si è votato a un compratore
Usabilità	Negativo: errore di voto non è correggibile
Mitigazione usabilità	L'interfaccia richiede conferma esplicita prima dell'invio

18 Smart Contract VotingContract

Il presente protocollo integra Ethereum come livello di registro pubblico immutabile. Tutte le operazioni critiche del ciclo di vita dell'elezione – apertura, raccolta voti, chiusura e pubblicazione del risultato – vengono ancorate on-chain tramite un unico smart contract denominato `VotingContract`, deployato su Ethereum prima dell'inizio delle operazioni.

18.1 Ruolo e Perimetro

Lo smart contract non sostituisce la logica crittografica del protocollo (cifratura, firma, Shamir), che rimane off-chain. Il suo ruolo è esclusivamente di **registro immutabile e verificabile**: riceve da AR, AS e EC i dati già prodotti e firmati nelle rispettive fasi, li cristallizza on-chain, e applica regole di stato che impediscono operazioni fuori sequenza. Le proprietà garantite sono:

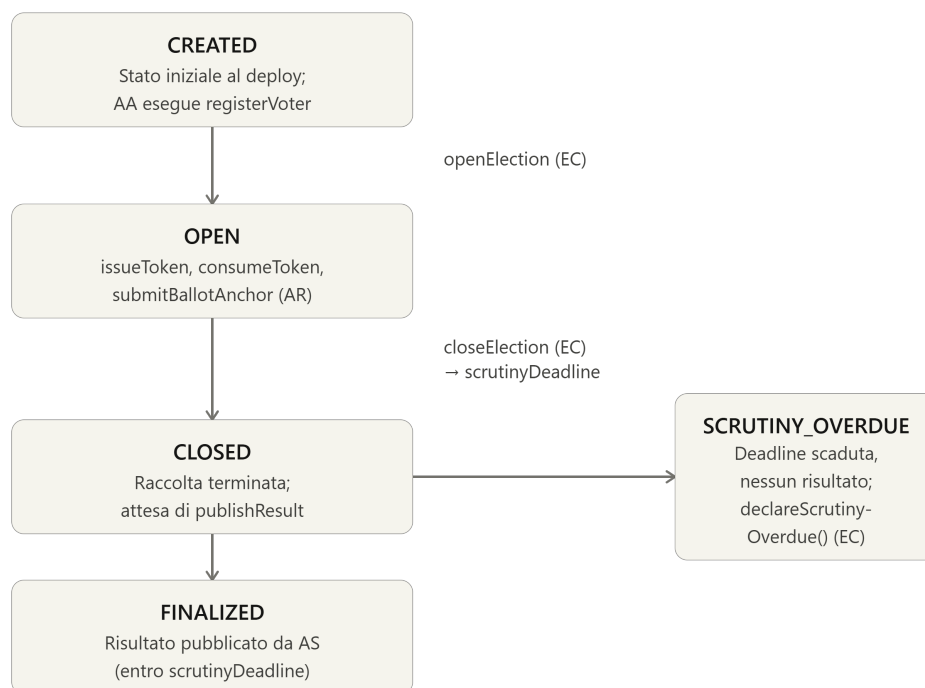
- **Immutabilità**: i dati scritti on-chain non possono essere modificati o cancellati da nessuna autorità.
- **Controllo di stato autonomo**: il contratto rifiuta operazioni non coerenti con la fase corrente (es. voti dopo la chiusura).
- **Verificabilità pubblica**: chiunque può leggere lo stato e gli eventi del contratto tramite qualsiasi nodo Ethereum o block explorer.

18.2 Macchina a Stati

Il contratto modella il ciclo di vita dell'elezione come una macchina a stati con cinque stati, ognuno dei quali abilita un sottoinsieme di funzioni:

Table 7: Stati del VotingContract

Stato	Descrizione e transizioni consentite
CREATED	Stato iniziale al deploy. <code>registerVoter</code> (solo AA) raccoglie le registrazioni degli elettori. Solo <code>openElection</code> (solo EC) transita allo stato successivo.
OPEN	Elezione aperta. Sono abilitate <code>issueToken</code> (solo AR), <code>consumeToken</code> (solo AR) e <code>submitBallotAnchor</code> (solo AR, include C_{voto} on-chain). <code>closeElection</code> (solo EC) chiude la raccolta e registra <i>scrutinyDeadline</i> .
CLOSED	Raccolta chiusa. <code>issueToken</code> , <code>consumeToken</code> e <code>submitBallotAnchor</code> sono bloccate. <code>publishResult</code> (solo AS) finalizza lo scrutinio, a condizione che $\text{block.timestamp} \leq \text{scrutinyDeadline}$. Trascorsa la deadline senza pubblicazione, <code>declareScrutinyOverdue()</code> (solo EC) transita allo stato SCRUTINY_OVERDUE.
FINALIZED	Risultato pubblicato. Nessuna scrittura ulteriore è consentita. Tutte le letture rimangono disponibili.
SCRUTINY_OVERDUE	Deadline di scrutinio scaduta senza pubblicazione del risultato. Nessuna scrittura ulteriore è consentita. Lo stato costituisce evidenza pubblica e immutabile del mancato adempimento di AS, azionabile sul piano istituzionale e legale.

**Figure 5:** Macchina a stati del VotingContract

19 Introduzione WP3

Il presente work package analizza la sicurezza del protocollo progettato in WP2 rispetto al modello di minaccia definito in WP1. Per ciascun avversario viene valutato l'esito degli attacchi che può portare e i meccanismi crittografici che vi si oppongono. Per ciascuna proprietà di sicurezza viene poi indicato lo stato di soddisfacimento, con motivazione esplicita dei limiti residui. L'analisi verifica altresì che le condizioni di completezza dichiarate in WP1 (Sezione 6) siano soddisfatte nel protocollo: le proprietà di sicurezza descrivono *cosa non deve accadere* in presenza di avversari, mentre le condizioni di completezza descrivono *cosa deve accadere* quando tutti gli attori operano onestamente; le due prospettive sono complementari.

Dove opportuno, vengono identificate modifiche migliorative non applicate nella soluzione attuale, con la motivazione della scelta effettuata, e discusso se tali modifiche produrrebbero benefici netti o comporterebbero perdite in altre proprietà.

L'analisi procede in tre fasi: (1) avversario per avversario (come in WP1, Sezione 3), con indicazione esplicita dell'esito di ciascun attacco; (2) proprietà per proprietà (come in WP1, Sezione 4), con tabelle di soddisfacimento per categoria; (3) analisi globale con assunzioni di fiducia residue e sintesi finale. Per ogni proprietà viene indicato lo stato: **garantita**, **parzialmente garantita** o **non garantita**.

20 Analisi per Avversario

20.1 Avversario Passivo (Eavesdropper)

L'avversario passivo intercetta il traffico di rete e tenta di correlare metadati (IP, timestamp, dimensione e frequenza dei messaggi) con l'identità dell'elettore senza alterare alcun dato.

Attacchi e relative mitigazioni

Il voto è cifrato con RSA-OAEP verso pk_{AS} : la CPA-security di OAEP garantisce che due cifrature dello stesso voto producano cifrati statisticamente indistinguibili. Il commitment $Commit_{voto} = SHA-256(v || r_{commit})$ non rivela alcuna informazione su v poiché r_{commit} non è trasmesso nella scheda: la proprietà di *hiding* è garantita dalla resistenza alla preimmagine di SHA-256. *Esito*: l'attacco fallisce — il contenuto del voto non è recuperabile da un avversario passivo.

Valutazione – Avversario Passivo

C.4 (protezione dall'intercettazione): **garantita** il contenuto è protetto da RSA-OAEP e firma.

C.2 (pseudo-anonimato): **parzialmente garantita** i metadati di rete rimangono esposti. Compromesso dichiarato.

20.2 Gestore Disonesto dell'Autorità di Raccolta (AR)

Attacchi e relative mitigazioni

Alterazione di voti registrati. Ogni scheda è firmata dall'elettore con sk_e ; qualsiasi modifica a C_{voto} o $Commit_{\text{voto}}$ invalida σ_{scheda} .

L'ancoraggio on-chain $H(token_{\text{sessione}}, H(B), Commit_{\text{voto}}, C_{\text{voto}}, ts_{\text{registrazione}})$, scritto dalla funzione `submitBallotAnchor`, è immutabile: qualsiasi alterazione successiva produce un $H(B)$ diverso, rilevabile dal confronto con la blockchain.

Esito: l'attacco fallisce la firma dell'elettore e l'ancoraggio on-chain rendono ogni modifica immediatamente rilevabile.

Soppressione di voti. La soppressione è rilevabile dall'elettore: se seq_{num} non è presente nell'array `ballotAnchors` on-chain, l'elettore ha prova della soppressione. **Esito: l'attacco è rilevabile** ma non prevenuto autonomamente richiede che l'elettore esegua la verifica individuale.

Omissione selettiva di `submitBallotAnchor`. AR potrebbe accettare una scheda off-chain e omettere deliberatamente la chiamata on-chain. La discrepanza tra la dimensione di `ballotAnchors` on-chain e N dichiarato alla chiusura è rilevabile da qualsiasi verificatore. **Esito: l'attacco è rilevabile** a livello universale; la prevenzione richiederebbe più AR indipendenti con consenso.

Violazione del pseudo-anonimato via metadati. AR conosce $H(Cert_e)$ presentato in fase di autenticazione e, poiché $Cert_e$ contiene ID_{elettore} , può associare l'identità alla scheda ricevuta. La soluzione completa (blind signature) è indicata come estensione futura. **Esito: l'attacco va a buon fine** rispetto al solo pseudo-anonimato verso AR limite dichiarato.

Valutazione – Gestore disonesto AR

I.1 (integrità post-registrazione): **garantita** firma elettore + ancoraggio on-chain.

I.3 (unicità voto): **garantita** `usedTokens` on-chain, non aggirabile da AR.

C.2 (pseudo-anonimato verso AR): **non garantita** AR può correlare identità e token. Compromesso dichiarato.

20.3 Gestore Disonesto dell'Autorità di Scrutinio (AS)

Attacchi e relative mitigazioni

Decifrazione anticipata dei voti individuali. AS non dispone di sk_{AS} fino alla fase di scrutinio (la chiave è distribuita in share ai garanti tramite Shamir). Con $t = 2$ su $n = 3$, la conoscenza di una sola condivisione non consente la ricostruzione: la sicurezza dello schema di Shamir garantisce che meno di t share non rivelino alcuna informazione su sk_{AS} . **Esito: l'attacco fallisce** se al più $t - 1 = 1$ garante colluda con AS.

Correlazione voto-identità tramite ordinamento temporale. AS recupera dalla blockchain $\{C_{\text{voto}_i}\}$ con il relativo indice seq_{num} , che riflette l'ordine temporale in cui AR ha ancorato le schede. In collusione con AR, AS potrebbe tentare di correlare l'indice con i metadati temporali noti ad AR. **Esito: l'attacco è parzialmente fattibile** in caso di collusione AR+AS, limitato al solo ordinamento temporale. Un shuffle verificabile (estensione futura) eliminerebbe questa correlazione.

Alterazione del risultato finale. Il risultato $R = \sum v_i$ è verificabile universalmente: chiunque può ricalcolarlo dalla lista $\{v_i\}$ nel BB. La firma $\sigma_{AS} = \text{Sign}_{sk_{AS}^{\text{sign}}}(R || N || \text{MerkleRoot}_{BB})$ vincola AS a un risultato specifico; qualsiasi discrepanza è immediatamente rilevabile e AS non può disconoscere la firma. **Esito: l'attacco fallisce** la verifica universale e la firma non ripudiabile rendono ogni alterazione rilevabile.

Valutazione – Gestore disonesto AS

C.1 (segretezza voto): **garantita** se meno di $t = 2$ garanti colludono con AS.

I.4 (risultato corretto): **garantita** tramite verifica universale e firma σ_{AS} non ripudiabile.

C.2 (pseudo-anonimato verso AS colludente con AR): **parzialmente garantita** correlazione limitata all'ordinamento temporale.

20.4 Autorità di Autenticazione Disonesta (AA)**Attacchi e relative mitigazioni**

Rilascio di credenziali duplicate. Il mapping `registeredVoters` on-chain impedisce la registrazione duplicata dello stesso elettore: `registerVoter` verifica l'assenza di $H(ID_{\text{elettore}})$ prima di procedere e rifiuta automaticamente la transazione in caso di duplicato. Il controllo è legato all'identità stabile dell'elettore e non al certificato, quindi un elettore non può ottenere un secondo certificato semplicemente generando una nuova coppia di chiavi. **Esito: l'attacco fallisce** il meccanismo on-chain non è aggirabile da AA.

Emissione multipla di token (vettore aggiuntivo). AA (o AR in collusione) potrebbe tentare di autenticare più volte lo stesso elettore — con nonce e timestamp freschi a ogni richiesta — per ottenere più token validi, da cedere a terzi o da usare per esprimere voti aggiuntivi. La contromisura è il mapping `issuedTokens` on-chain: la funzione `issueToken` del contratto verifica che $H(Cert_e)$ non sia già presente e rifiuta qualsiasi seconda invocazione per lo stesso certificato, rendendo il tentativo pubblicamente tracciabile sulla blockchain. **Esito: l'attacco fallisce** `issuedTokens` vincola a un unico token per certificato, indipendentemente dal numero di autenticazioni.

Rilascio di credenziali a non aventi diritto. Il contratto non può verificare l'identità reale: l'autenticità è demandata al processo istituzionale fuori banda. **Esito: l'attacco va a buon fine** se AA è compromessa limite dichiarato, assunzione di fiducia residua verso AA.

Tracciamento delle preferenze in collusione con AR. AA conosce $Cert_e$ rilasciato a ogni elettore; colludendo con AR (che vede il token derivato da $Cert_e$) potrebbe correlare identità e preferenze. **Esito: l'attacco va a buon fine** in caso di collusione AA+AR limite condiviso con C.2.

Denial-of-service selettivo. AA potrebbe rifiutare di emettere certificati a specifici elettori. Non esistono contromisure crittografiche; la trasparenza on-chain consente di dimostrare che un elettore non sia mai stato registrato, fornendo evidenza di rifiuto ingiustificato. **Esito: l'attacco va a buon fine** sul piano tecnico mitigazione solo operativa/legale.

Valutazione – AA disonesta

I.2 (solo aventi diritto): **parzialmente garantita** anti-duplicato on-chain; legittimità identità fiduciaria.

C.3 (dati personali non trasmessi ad AS): **garantita** per struttura del protocollo.

E.1 (accessibilità): **non garantita contro DoS selettivo** - contromisura operativa, non crittografica.

20.5 Avversario Esterno**Attacchi e relative mitigazioni**

Impersonazione di un elettore. Per farlo l'avversario dovrebbe possedere sia sk_e sia $Cert_e$. La sottrazione di sk_e richiede la compromissione del dispositivo dell'elettore. **Esito: l'attacco fallisce** in assenza di compromissione del dispositivo.

Replay attack. Il nonce e il timestamp nella fase di autenticazione (Equazione (4)) e nella scheda ($nonce_{\text{voto}}, ts_{\text{voto}}$) rendono ogni messaggio univoco. Il controllo $|ts - ts_{\text{corrente}}| < \Delta_{\text{max}}$ impedisce l'uso ritardato; il registro locale off-chain dei nonce gestito da AR previene il replay nell'intervallo di validità del timestamp. La garanzia permanente inter-sessione è fornita dai mapping on-chain `issuedTokens` (unicità del token per certificato) e `usedTokens` (unicità del consumo del token). *Esito:* l'attacco fallisce — i meccanismi anti-replay operano su più livelli complementari.

Man-in-the-Middle sulla distribuzione delle chiavi. La PKI con certificati X.509 firmati da CA previene attacchi MitM: la chiave pubblica di un'autorità è autentica se e solo se il certificato è verificabile tramite pk_{CA} . Le chiavi Ethereum sono vincolate all'identità istituzionale tramite estensioni X.509. **Esito:** l'attacco fallisce la PKI istituzionale costituisce il trust anchor.

Denial-of-service sull'infrastruttura. Un attacco DoS contro AR durante la finestra di voto impedirebbe ad alcuni elettori di votare. Il protocollo non include meccanismi di resilienza DoS. **Esito: l'attacco va a buon fine.**

Valutazione – Avversario esterno

C.4 (protezione in transito): **garantita** tramite cifratura RSA-OAEP e firma.

I.5 (credenziali non falsificabili): **garantita** da PKI + RSA-FDH.

Replay: garantita tramite nonce + timestamp (off-chain) + `issuedTokens/usedTokens` on-chain.

DoS: **non garantita** contromisura operativa, non crittografica.

20.6 Elettore Disonesto e Coercitore

Attacchi e relative mitigazioni

Voto multiplo. Il meccanismo `issueToken` on-chain garantisce che per ogni $H(Cert_e)$ sia emesso al più un token; `consumeToken` garantisce che ogni token sia consumabile al più una volta. Entrambi i controlli sono on-chain e non aggirabili nemmeno da AR. **Esito: l'attacco fallisce** il doppio voto è impossibile anche con più tentativi di autenticazione con nonce diversi.

Vendita del voto. L'elettore potrebbe rivelare r_{commit} a un compratore dopo la pubblicazione del BB, consentendo di verificare $\text{SHA-256}(v \| r_{\text{commit}}) = \text{Commit}_{\text{voto}}$ e dunque dimostrare il valore di v . *Esito:* l'attacco va parzialmente a buon fine — è una limitazione strutturale del protocollo, dichiarata in C.6.

Coercizione. Un coercitore presente fisicamente durante la sessione potrebbe imporre la scelta del voto. La mancanza di un meccanismo di sostituzione del voto (C.7) è una scelta deliberata: introdurre la revocabilità consentirebbe al coercitore di imporre la ri-espressione, aggravando la coercizione invece di mitigarla. **Esito: la coercizione in fase di espressione non ha contromisure crittografiche** limite strutturale del voto remoto, dichiarato in WP1.

Valutazione – Elettore disonesto e coercitore

I.3 (unicità voto): **garantita** on-chain, non aggirabile nemmeno con più autenticazioni.

C.6 (resistenza coercizione): **parzialmente garantita** l'elettore può rivelare r_{commit} post-pubblicazione.

C.7 (non-revocabilità): **garantita** scelta deliberata a favore della resistenza alla coercizione.

21 Analisi per Proprietà

Nella presente sezione si ricapitola sistematicamente lo stato di ciascuna proprietà dichiarata in WP1. Per ogni categoria viene riportata una tabella di soddisfacimento e una breve discussione discorsiva.

21.1 Confidenzialità

Table 8: Grado di soddisfacimento delle proprietà di confidenzialità

Prop.	Descrizione	Stato	Meccanismo / Limite
C.1	Segretezza voto	Garantita	RSA-OAEP (CPA-secure) + hiding del commitment + distruzione sk_{AS} post-scrutinio.
C.2	Pseudo-anonimato	Parziale	AR può correlare identità e token tramite $Cert_e$.
C.3	Isolamento dati personali	Garantita	Per struttura del protocollo AA non trasmette ID_{elettore} ad AR né ad AS.
C.4	Protezione in transito	Garantita	Cifratura RSA-OAEP + firma RSA-FDH su ogni messaggio.
C.5	Credenziale anonima	Parziale	Il token contiene pk_e (estratta da $Cert_e$ in fase di autenticazione); AR può risalire all'identità dell'elettore tramite il $Cert_e$ ricevuto nella stessa fase.
C.6	Resistenza coercizione	Parziale	L'elettore può rivelare r_{commit} dopo la pubblicazione del BB. Compromesso dichiarato.
C.7	Non-revocabilità	Garantita	Scelta progettuale deliberata a favore della resistenza alla coercizione.

21.2 Integrità

Table 9: Grado di soddisfacimento delle proprietà di integrità

Prop.	Descrizione	Stato	Meccanismo / Limite
I.1	Immutabilità voti	Garantita	Firma σ_{scheda} + ancoraggio on-chain immutabile tramite <code>submitBallotAnchor</code> .
I.2	Solo aventi diritto	Parziale	Anti-duplicato on-chain; legittimità dell'identità demandata al processo istituzionale fuori banda.
I.3	Unicità voto	Garantita	<code>issuedTokens</code> + <code>usedTokens</code> on-chain, entrambi append-only e non aggirabili nemmeno da AR.
I.4	Correttezza risultato	Garantita	Verifica universale $R = \sum v_i$; firma σ_{AS} non ripudiabile.
I.5	Credenziali autentiche	Garantita	PKI X.509 + RSA-FDH; non falsificabili sotto assunzione RSA.
I.6	No voti fraudolenti	Parziale	Dipende da I.2: vedi limite su AA per la verifica dell'identità.
I.7	Verificabilità a posteriori	Garantita	Tutti i dati critici ancorati on-chain; eventi Ethereum immutabili e pubblicamente ispezionabili.

21.3 Trasparenza

Table 10: Grado di soddisfacimento delle proprietà di trasparenza

Prop.	Descrizione	Stato	Meccanismo / Limite
T.1	Protocollo pubblico	Garantita	Protocollo documentato; algoritmi standard (RSA-OAEP, RSA-FDH, SHA-256, X.509, Shamir). La demo implementa Shamir in <code>shamir_setup.py</code> ; i garanti sono simulati in-process anziché essere entità fisicamente separate (si veda WP4).
T.2	Verificabilità universale	Garantita	$R, \sigma_{AS}, Att_{risultato}, MerkleRoot_{BB}$ leggibili on-chain senza credenziali.
T.3	Verificabilità individuale	Garantita	Verifica tramite seq_{num} , ricevuta e BB; accessibile direttamente on-chain.
T.4	Trust minimale	Garantita	Ethereum riduce il trust verso le autorità per la pubblicazione; assunzioni residue dichiarate.
T.5	Attestazioni legali	Garantita	$Att_{apertura}, Att_{chiusura}$ e $Att_{risultato}$ ancorati on-chain in modo immutabile tramite <code>openElection, closeElection</code> e <code>publishResult</code> .

21.4 Efficienza

Table 11: Grado di soddisfacimento delle proprietà di efficienza

Prop.	Descrizione	Stato	Meccanismo / Limite
E.1	Voto in tempi ragionevoli	Parziale	Transazione on-chain in consumeToken introduce latenza 12–30 s; accettabile per elezione non ad alta frequenza. Rischio aggiuntivo: DoS selettivo da AA disonesta o avversario esterno contro l'infrastruttura AR, senza contromisure crittografiche.
E.2	Verifica individuale semplice	Parziale	Tecnicamente accessibile via block explorer; richiede strumento dedicato per utenti non esperti.
E.3	Verifica universale	Garantita	Somma di N bit + controllo firma + interrogazione block explorer con risorse ordinarie.
E.4	Dimensione messaggi	Garantita	Scheda di voto: $O(1)$ blocchi RSA-OAEP; commitment e firma in $O(1)$.

22 Analisi Globale del Sistema

22.1 Assunzioni di Fiducia Residue

Il protocollo riduce al minimo le assunzioni di fiducia ma non le elimina completamente. La Tabella 12 riassume le assunzioni residue e il rischio associato.

Table 12: Assunzioni di fiducia residue e rischio

Entità	Assunzione	Rischio in caso di violazione
AA	Verifica correttamente le identità e non emette certificati a non aventi diritto	Voti fraudolenti nel conteggio; on-chain rileva solo duplicati, non la legittimità dell'identità
Garanti AS (al più $t - 1$ disonesti)	Al più $t - 1 = 1$ garante colluda con AS per la ricostruzione anticipata di sk_{AS}	Decifratura anticipata dei voti individuali durante la raccolta
CA	Emette certificati solo dopo verifica dell'identità istituzionale	Chiave pubblica di un avversario certificata come autorità legittima
Ethereum	La rete non subisce attacchi al consenso (51% attack) durante la finestra critica	Riscrittura degli ancoraggi on-chain; altamente improbabile su Mainnet

22.2 Conflitti tra Proprietà

La progettazione ha richiesto di bilanciare proprietà strutturalmente in tensione fra loro. I principali compromessi adottati sono ricapitolati nella Tabella 13.

Table 13: Compromessi tra proprietà nel protocollo

Proprietà privilegiata	Proprietà sacrificata	Motivazione
Resistenza alla coercizione (C.6, C.7)	Usabilità (correggere errori)	La revocabilità consentirebbe coercizione post-voto; la non-revocabilità è il compromesso standard in letteratura.
Verificabilità individuale (T.3)	Pseudo-anonimato (C.2)	Il seq_{num} è correlabile al timestamp di registrazione on-chain in collusione AR+AS.
Efficienza (E.1)	Decentralizzazione immediata	Le transazioni on-chain introducono latenza (12–30 s); tollerata per garantire integrità e unicità in modo autonomo.

22.3 Sintesi

Table 14: Riepilogo dello stato di tutte le proprietà

Prop.	Descrizione sintetica	Stato	Note
C.1	Segretezza voto	Garantita	RSA-OAEP + distruzione sk_{AS}
C.2	Pseudo-anonimato	Parziale	Limite verso AR colludente
C.3	Isolamento dati personali	Garantita	Strutturale
C.4	Protezione in transito	Garantita	RSA-OAEP + firma
C.5	Credenziale anonima	Parziale	Token tracciabile da AR
C.6	Resistenza coercizione	Parziale	Reveal di r_{commit} possibile
C.7	Non-revocabilità	Garantita	Scelta deliberata
I.1	Immutabilità voti	Garantita	Firma + ancoraggio on-chain
I.2	Solo aventi diritto	Parziale	Fiducia residua in AA
I.3	Unicità voto	Garantita	issuedTokens + usedTokens
I.4	Correttezza risultato	Garantita	Verifica universale
I.5	Credenziali autentiche	Garantita	PKI + RSA-FDH
I.6	No voti fraudolenti	Parziale	Dipende da AA
I.7	Verificabilità a posteriori	Garantita	Ethereum immutabile
T.1	Protocollo pubblico	Garantita	Algoritmi standard
T.2	Verificabilità universale	Garantita	Dati on-chain pubblici
T.3	Verificabilità individuale	Garantita	Receipt + on-chain
T.4	Trust minimale	Garantita	Ethereum riduce trust
T.5	Attestazioni legali	Garantita	EC on-chain
E.1	Voto in tempi ragionevoli	Parziale	Latenza on-chain 12–30 s
E.2	Verifica individuale semplice	Parziale	Richiede strumento dedicato
E.3	Verifica universale	Garantita	Somma + firma + block explorer
E.4	Dimensione messaggi	Garantita	$O(1)$ per scheda

WORK PACKAGE 4 — Implementazione e Benchmark

Il codice sorgente del prototipo è disponibile nella repository GitHub: https://github.com/fabriziodrr/APS_projectwork_group2.

23 Strumenti Crittografici Utilizzati

Il prototipo fa uso di un insieme ristretto di primitive crittografiche consolidate, ciascuna scelta per garantire proprietà di sicurezza precise rispetto al modello di minaccia definito nel WP2.

23.1 RSA-2048 e PKI X.509

Tutte le coppie di chiavi asimmetriche sono RSA a 2048 bit con esponente pubblico $e = 65537$. La PKI è basata su certificati X.509v3: la CA radice firma i certificati di AA, AR, AS ed EC tramite `issue_authority_certificate`; AA firma i certificati degli elettori ($Cert_e$). La catena di fiducia è verificabile autonomamente senza infrastruttura esterna.

23.2 RSA-OAEP (Cifratura del Voto)

La cifratura del bit di voto avviene tramite **RSA-OAEP** con SHA-256 e MGF1-SHA-256 (PKCS#1 v2.2). Lo schema garantisce sicurezza CPA e non-malleabilità. Il plaintext ha forma $(v || r_{\text{voto}})$ con r_{voto} a 16 byte, rendendo ogni ciphertext statisticamente indistinguibile dagli altri. La stessa primitiva cifra la chiave AES nel protocollo di trasmissione ibrida delle share Shamir.

23.3 RSA-PKCS1v15 + SHA-256 (Firma Digitale)

Le firme digitali su token AR, schede di voto, ricevute, Bulletin Board e pacchetti share usano RSA-PKCS1v15 con SHA-256. La firma è calcolata sull'hash del messaggio ($\sigma = \text{Sign}_{sk}(H(m))$), approssimando didatticamente RSA-FDH (*Full Domain Hash*). Questa primitiva garantisce la non-ripudiabilità delle ricevute AR e l'integrità del Bulletin Board.

23.4 SHA-256 (Hash e Commitment)

SHA-256 è usata per tre scopi distinti:

1. **Identificazione:** $H(ID_e)$ registrato on-chain da AA; $H(Cert_e)$ come chiave in `issuedTokens`; $H(B)$ come identificativo della scheda.
2. **Commitment al voto:** $\text{Commit}_{\text{voto}} = \text{SHA-256}(v || r_{\text{commit}})$ con r_{commit} a 32 byte. La proprietà di *binding* impedisce ad AS di alterare v mantenendo un commitment consistente; la proprietà di *hiding* impedisce di risalire al voto dal solo commitment pubblico on-chain.
3. **Digest intermedi nella firma:** ogni firma opera su $H(m)$ per uniformare la lunghezza dell'input al modulo RSA.

23.5 AES-256-GCM (Cifratura Simmetrica delle Share)

La trasmissione delle share Shamir segue il paradigma *encrypt-then-sign* con cifratura ibrida: la chiave AES a 256 bit viene cifrata con RSA-OAEP, mentre il payload della share è cifrato con **AES-256-GCM**

(AEAD). Il tag MAC a 128 bit di GCM garantisce integrità e riservatezza del ciphertext senza necessità di un MAC separato.

23.6 Schema di Shamir ($t=2, n=3$) su $\text{GF}(p)$

La chiave di decifratura sk_{AS} non viene mai scritta su disco. L'esponente privato d è il segreto S suddiviso secondo Shamir ($t=2, n=3$) su $\text{GF}(p)$, con $p > d$ primo generato via Miller-Rabin ($k=20$ testimoni, probabilità di falso-positivo $\leq 4^{-20}$). Il polinomio è $f(x) = S + a_1x \bmod p$; le share sono $s_i = f(i)$. La ricostruzione usa l'interpolazione di Lagrange; da d ricostruito, sk_{AS} viene rigenerata in memoria fattorizzando n tramite l'algoritmo che sfrutta $k = de - 1$.

24 Architettura Software: Ruoli degli Script Python

Il prototipo si articola in undici moduli Python principali, un contratto Solidity e due script di orchestrazione.

`config.py` Centralizza tutti i parametri condivisi: percorsi PKI, costanti crittografiche (`RSA_KEY_SIZE`, finestre anti-replay, dimensioni dei fattori di randomizzazione), indirizzi Ethereum Ganache e percorsi di compilazione Solidity.

`crypto_utils.py` Raccoglie tutte le primitive crittografiche condivise: RSA-OAEP, firma/verifica PKCS1v15+SHA-256, commitment SHA-256, serializzazione canonica dei messaggi, e le primitive per la trasmissione sicura delle share (`encrypt_share`, `decrypt_share`, `sign_share_packet`, `verify_share_packet`).

`ca.py` Trust anchor della PKI. Genera il certificato radice self-signed e firma i certificati X.509 delle autorità e degli elettori.

`aa.py` Registra gli elettori calcolando $H(ID_e)$, invocando `registerVoter` on-chain per prevenire doppi ingressi, ed emettendo $Cert_e = \text{Sign}_{sk_{AA}}(pk_e || ID_e)$. Non partecipa a voto né scrutinio.

`ar.py` Autentica gli elettori (verifica $Cert_e$, nonce, finestra temporale), emette il token firmato e raccoglie le schede. Esegue tutti i controlli crittografici off-chain prima di qualsiasi operazione on-chain per prevenire attacchi DoS.

`as_.py` Esegue lo scrutinio: raccoglie le share dai garanti via `encrypt-then-sign`, verifica `MerkleRootUrna`, decifra i voti con sk_{AS} ricostruita in memoria (poi distrutta), firma il Bulletin Board con sk_{AS}^{sign} e pubblica il risultato on-chain.

`shamir_setup.py` Implementa l'intera matematica Shamir: Miller-Rabin per la ricerca del primo p , split e interpolazione di Lagrange su $\text{GF}(p)$, fattorizzazione probabilistica di n per ricostruire `RSAPrivateKey` (con CRT).

`ec.py` Ente Certificatore Legale: firma e registra on-chain le attestazioni di apertura, chiusura e risultato. Può dichiarare `SCRUTINY_OVERDUE` in caso di inadempienza di AS.

`elettore.py` Client lato utente: genera (pk_e, sk_e) localmente, si registra presso AA, si autentica ad AR con nonce e σ_e , costruisce la scheda (OAEP + commitment + firma σ_{scheda}) e conserva i dati per la verifica individuale.

`verifica.py` Verifica individuale (ancoraggio on-chain, autenticità ricevuta, apertura commitment) e verifica universale (coerenza $AR \leftrightarrow AS$, somma voti, firma BB, attestazione EC). Non richiede credenziali private.

`blockchain.py` Interfaccia Web3 per Ganache: compila e deploia il contratto Solidity, espone wrapper Python per tutte le funzioni on-chain, implementa le funzioni Merkle.

`run_voting.py` / `start_all.py` Orchestrazione del flusso completo. `run_voting.py` assume il setup già completato; `start_all.py` include anche setup e registrazione degli elettori. Entrambi gestiscono riesecuzioni parziali saltando le fasi già completate.

`gui.py` Interfaccia web (Flask, porta 5050) con job asincroni per le transazioni Ganache; il frontend fa polling ogni 500 ms su `/api/job/<id>` per evitare timeout HTTP.

25 Analisi dei Tempi di Esecuzione

I tempi riportati nella Tabella 15 sono stati misurati eseguendo lo script `benchmark.py` incluso nel prototipo su una macchina locale Windows 11 con processore Intel Core i7, chiavi RSA a 2048 bit, Python 3.12 e libreria `cryptography` 42.x. Ogni operazione è stata ripetuta tra 20 e 50 volte. Le righe grigie indicano operazioni di *setup* (una tantum pre-elezione), che non impattano la latenza dell'elettore.

Table 15: Tempi medi delle operazioni crittografiche del sistema APS (RSA-2048, benchmark locale, `benchmark.py`).

Operazione	Media (ms)	Commento
Hashing SHA-256		
SHA-256 su 64 byte	< 0,01	Trascurabile. Usato per digest di token, schede e ricevute.
Commitment voto $SHA-256(v r)$	< 0,01	Istantaneo. Il fattore r_{commit} a 32 byte garantisce hiding e binding.
Firma e verifica RSA-PKCS1v15 + SHA-256		
Firma RSA-2048 (sk_{AR}, sk_e)	≈ 0,9	Dominata dall'esponenziazione CRT. Usata per token AR, scheda elettore e ricevuta.

continua nella pagina successiva...

Table 15 – continua dalla pagina precedente

Operazione	Media (ms)	Commento
Verifica firma RSA-2048	$\approx 0,05$	Circa $18\times$ più veloce della firma ($e = 65537$ piccolo). Eseguita da AR e AS in fase di controllo.
Cifratura asimmetrica RSA-OAEP		
Cifratura voto RSA-OAEP (pk_{AS})	$\approx 0,05$	Solo chiave pubblica. Eseguita una volta per voto dall'elettore.
Decifratura voto RSA-OAEP (sk_{AS})	$\approx 1,1$	Usa la chiave privata di AS. Con N voti: $N \times 1,1$ ms (parallelizzabile).
Cifratura simmetrica AES-256-GCM		
AES-256-GCM cifratura/decifratura	$< 0,01$	Trascurabile. Il tag GCM garantisce integrità senza MAC separato.
Trasmissione share Shamir (RSA-OAEP + AES-GCM + firma)		
Cifratura pacchetto share garante	$\approx 0,08$	Dominata da RSA-OAEP sulla chiave AES. Una per garante.
Decifratura pacchetto share	$\approx 0,9$	Dominata da RSA-OAEP (decifratura). AS esegue per $t=2$ garanti.
Protocollo di voto: token, scheda, ricevuta		
Costruzione token AR	$\approx 0,9$	Una firma RSA; emesso una volta per elettore.
Verifica token AR	$\approx 0,05$	Una verifica di firma; eseguita da AR alla ricezione scheda.
Costruzione scheda + firma σ_{scheda}	$\approx 0,9$	OAEP + commitment + firma RSA; eseguita dall'elettore una volta.
Verifica firma scheda	$\approx 0,07$	Trascurabile rispetto alla latenza della transazione Ethereum.

continua nella pagina successiva...

Table 15 – continua dalla pagina precedente

Operazione	Media (ms)	Commento
Costruzione ricevuta AR	$\approx 0,9$	Firma RSA su $(H(B), seqNum, ts)$; prova non ripudiabile.
PKI — Certificati X.509		
Emissione certificato X.509	$\approx 3,4$	Firma RSA del TBS certificate. Una volta per attore (setup).
Schema di Shamir — operazioni di setup (<i>una tantum</i>)		
Shamir split su $GF(p)$	$< 0,01$	Puro calcolo modulare; trascurabile.
Lagrange ricostruzione ($t=2$)	$\approx 0,03$	Interpolazione con due punti su $GF(p)$; immediata.
Miller-Rabin (256 bit, $k=20$)	$< 0,01$	Probabilità di falso-positivo $\leq 4^{-20}$; molto rapido.
Ricerca primo $p > S$ (256 bit)	≈ 13	Variabile in base al gap. In produzione (p a ~ 2048 bit) può richiedere 1–5 min, ma avviene una sola volta.
Generazione chiavi — setup (<i>una tantum</i>)		
Generazione coppia RSA-2048	≈ 50	Ricerca di due primi forti a 1024 bit. Setup PKI completo (~ 8 coppie): $\lesssim 1$ s in totale.

Interpretazione dei tempi

Le operazioni **in-protocollo** (firma, OAEP, verifica) sono tutte inferiori a 2 ms, rendendo la latenza crittografica lato-utente impercettibile: l'intero flusso di voto di un singolo elettore richiede < 10 ms di puro calcolo crittografico. Il collo di bottiglia reale nella demo è la transazione Ethereum su Ganache (50–200 ms per transazione). Le operazioni di setup evidenziate in grigio avvengono *una sola volta* in fase di configurazione pre-elezione.